

FANTASE - FANTastic SEquence and Where to Find Them: Faithful and Efficient API Call Generation through State-tracked Constrained Decoding and Reranking

Zhuoer Wang @ Texas A&M University

<https://edillower.github.io>

Outline

1. Background & Motivation
2. Reranking
3. State-tracked Constrained Decoding
4. Experiment Results
5. Conclusion

Background & Motivation

The API Call Generation Task



I want to find a **burger joint**.

In which city?



In **Mountain View**.

Eureka! restaurant is in Mountain View.



Are there any other restaurants in the **moderate price range**?

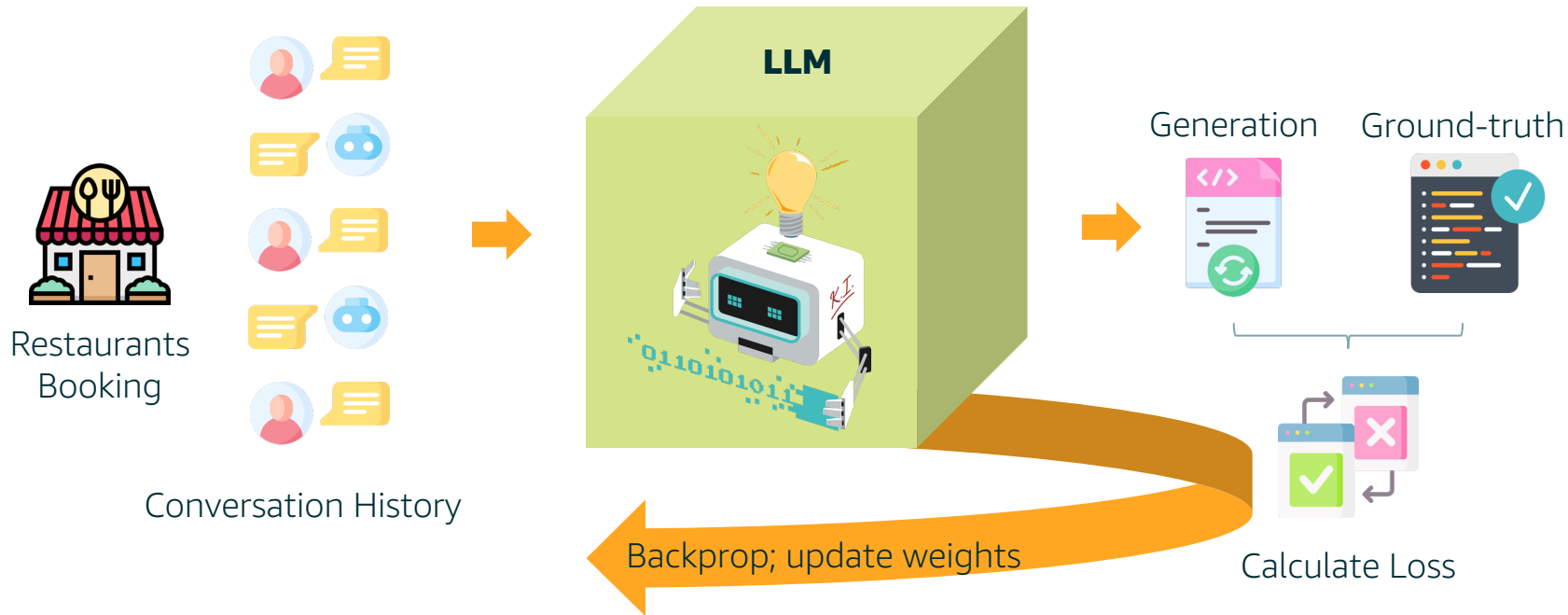
**Restaurants_1.FindRestaurants(city="Mountain View",
cuisine="American", price_range="moderate")**



Background & Motivation

Current Solution

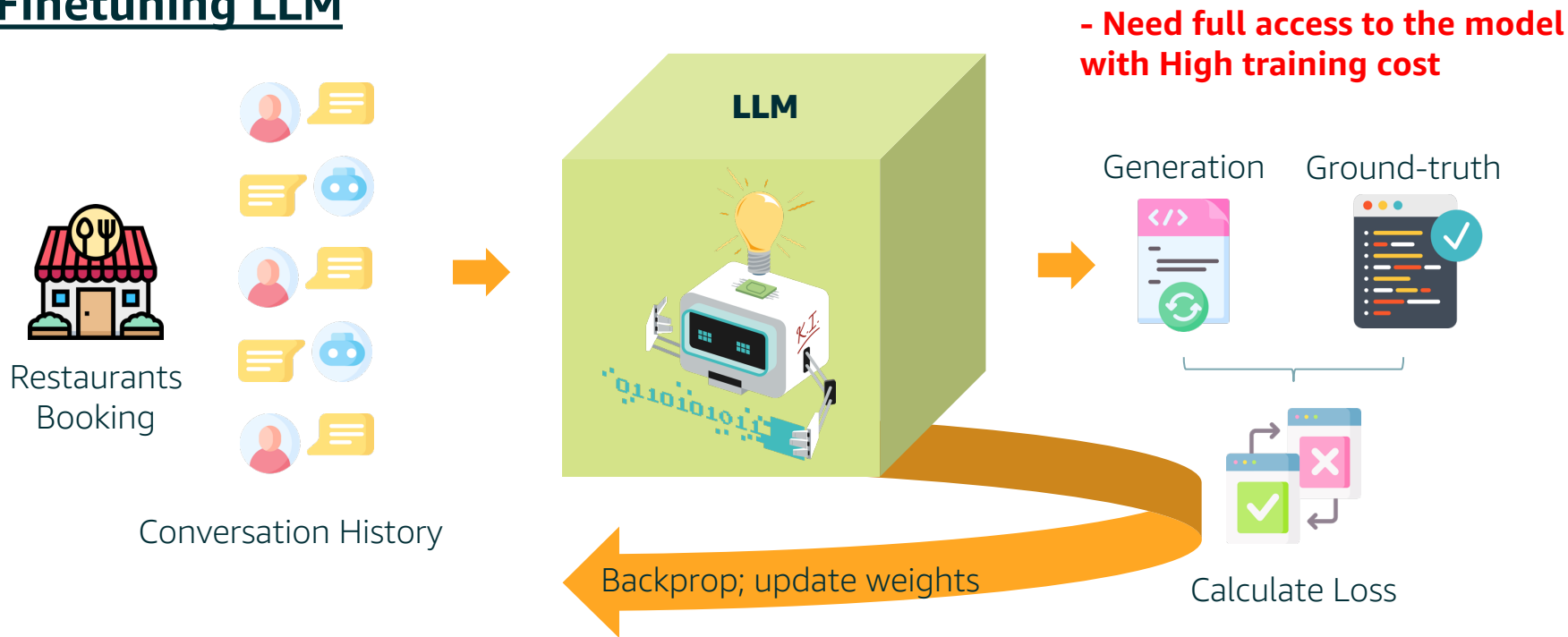
Finetuning LLM



Background & Motivation

Current Solution

Finetuning LLM



Background & Motivation

Current Solution

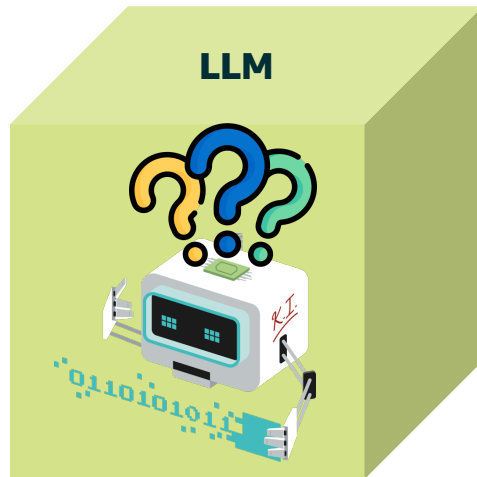
Finetuning LLM



Other Tasks



Updated API



- Need full access to the model with High training cost
- Hard adaptation to new API and context
- Loss of general knowledge

Restaurants_1.FindRestaurants("cuisine" : Required, [American, Chinese, Indian], "city" : Required)



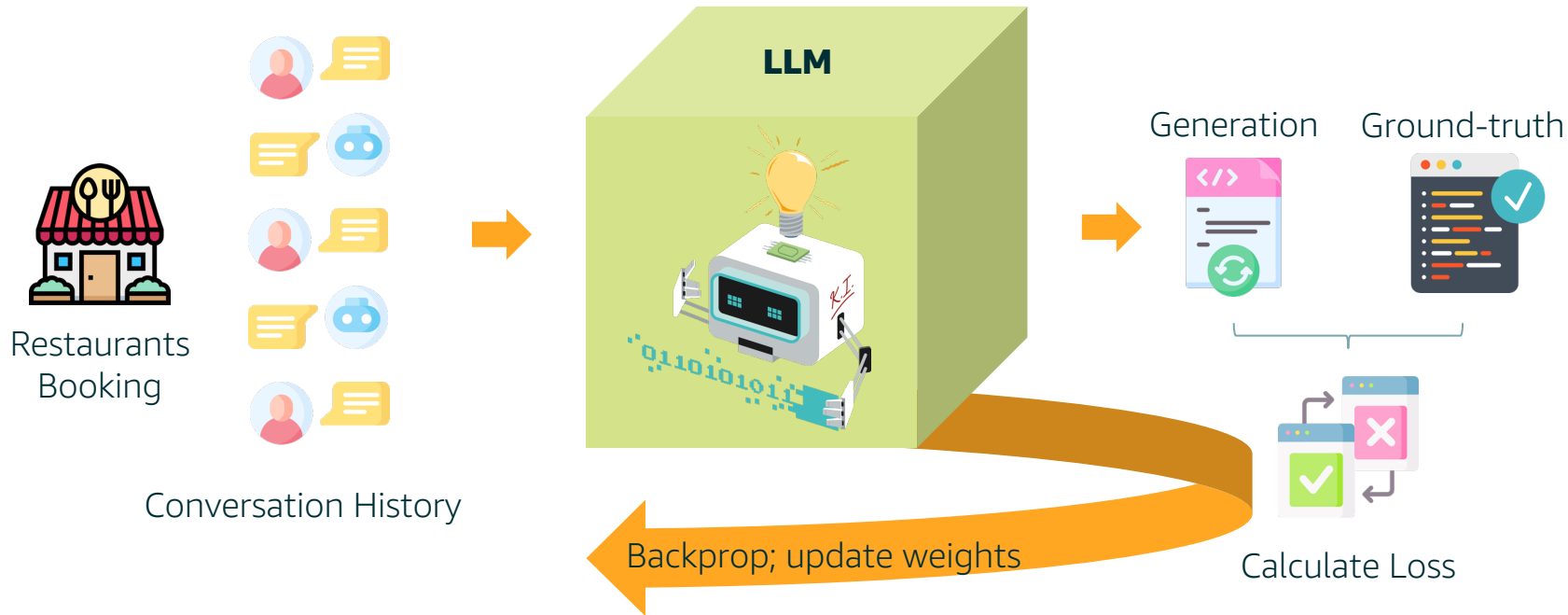
Merchant.SearchRestaurants("cuisine" : Required, "city" : Required, "country" : Required)



Background & Motivation

Current Solution

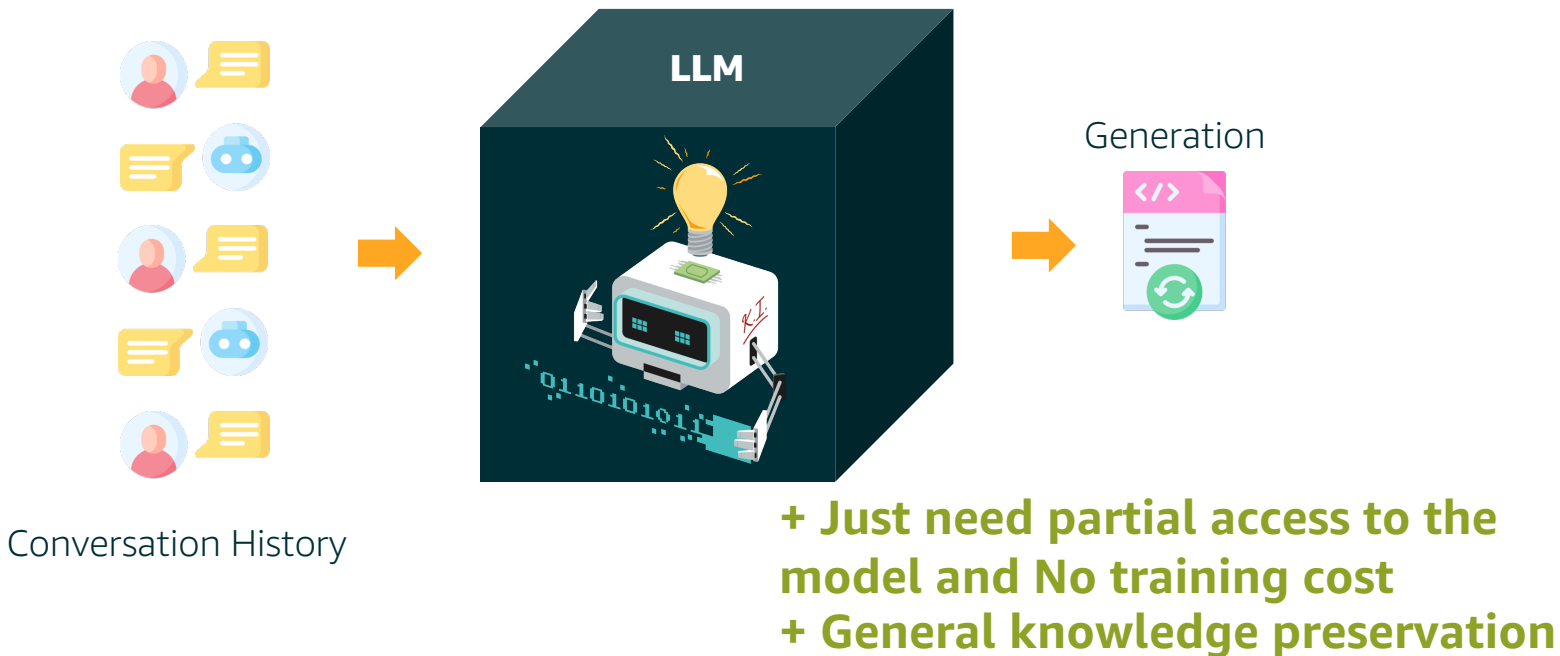
From Finetuning to Zero-shot/Few-shot In-context Learning



Background & Motivation

Current Solution

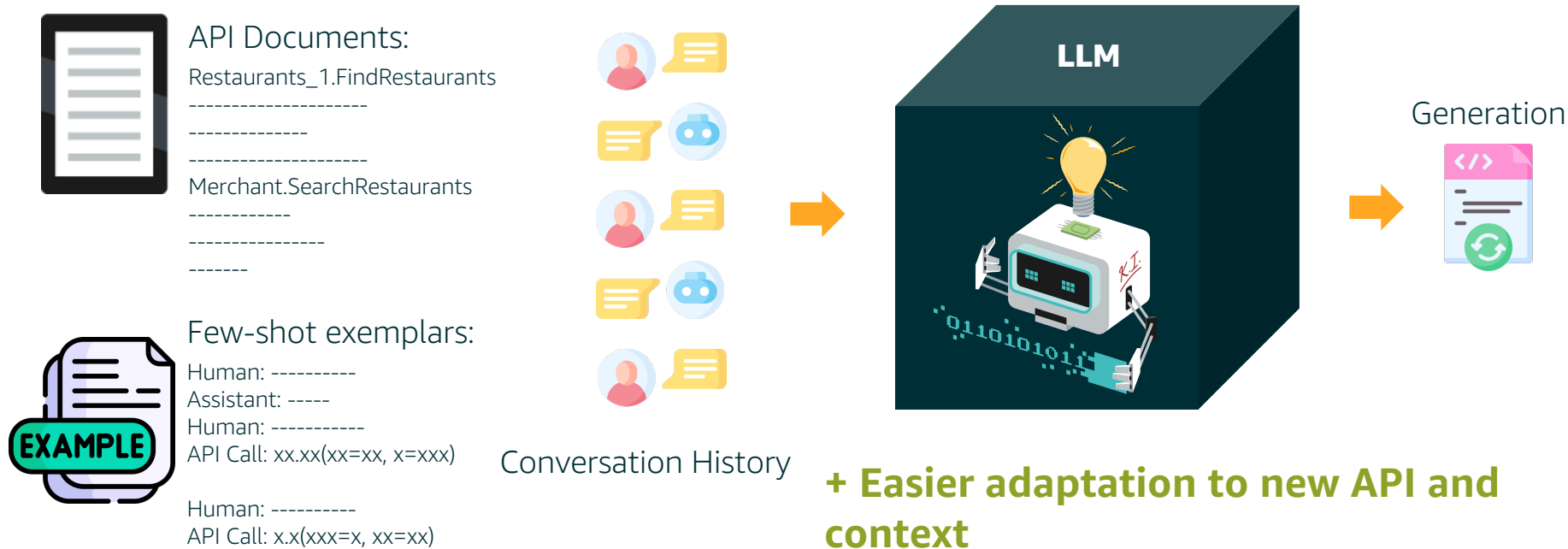
From Finetuning to Zero-shot/Few-shot In-context Learning



Background & Motivation

Current Solution

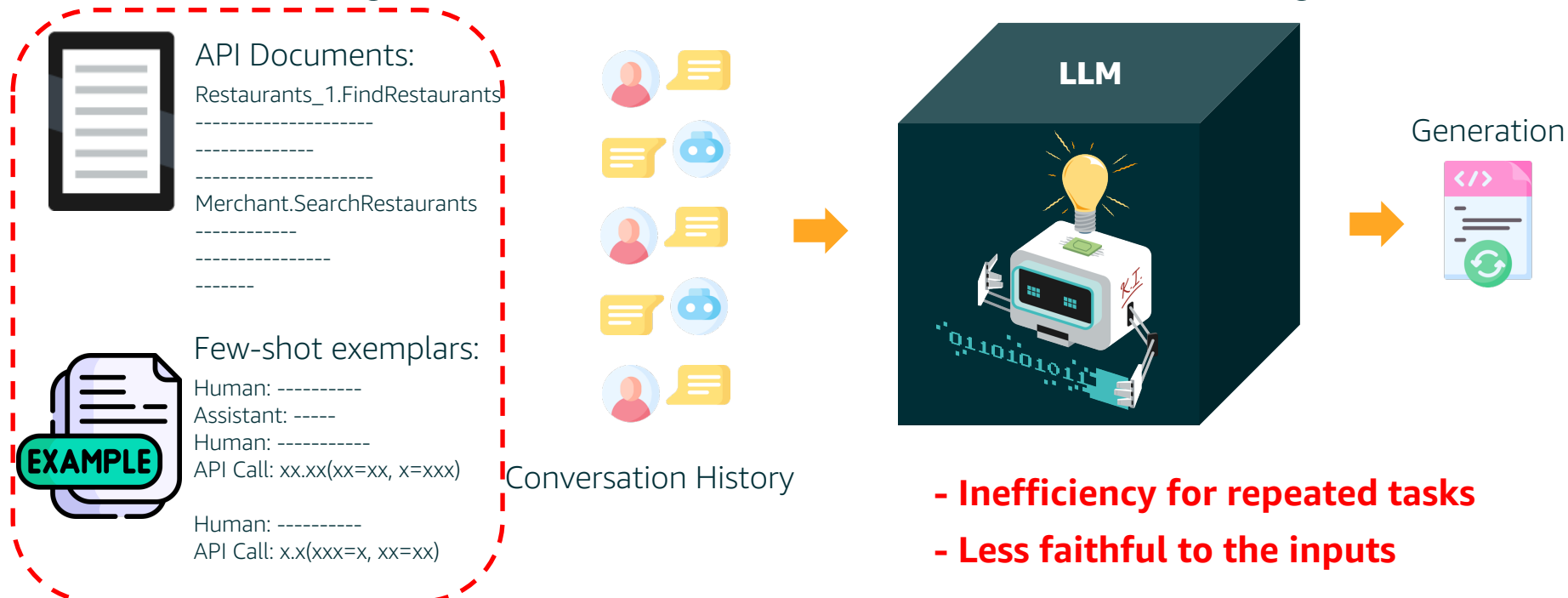
From Finetuning to Zero-shot/Few-shot In-context Learning



Background & Motivation

Current Solution

From Finetuning to Zero-shot/Few-shot In-context Learning

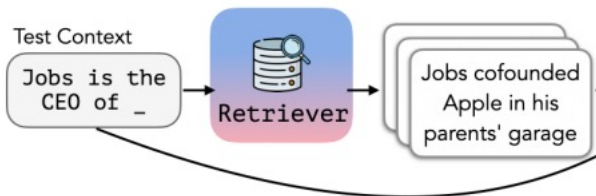


Background & Motivation

How to improve the generation with black-box LLM?

RE-PLUG 

Frozen/Trainable



Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Improve the prompts

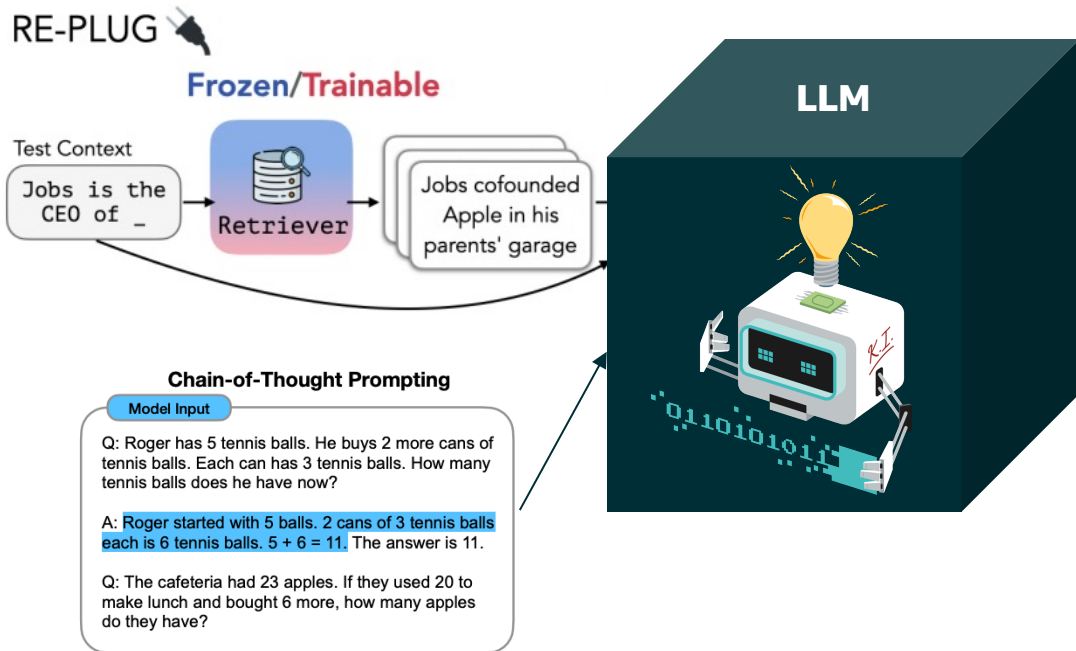
- Retrieval-augmented LLM
- Chain-of-Thought Prompting
- Prompt Engineering
- Exemplar Selection
- Improve API short-listener

.....



Background & Motivation

An often-forgetting and critical component of generation ...



Background & Motivation

Decoding!

RE-PLUG 

Frozen/Trainable

Test Context

Jobs is the
CEO of _



Jobs cofounded
Apple in his
parents' garage

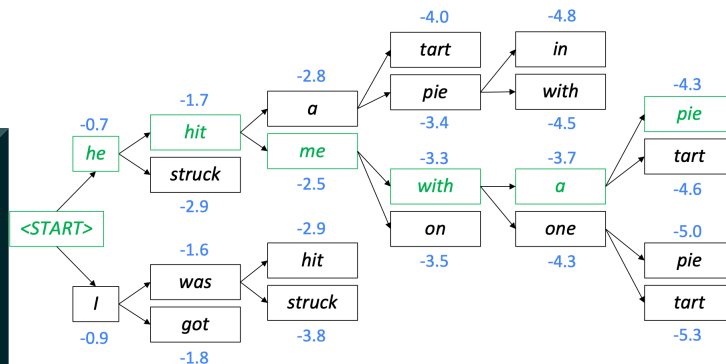
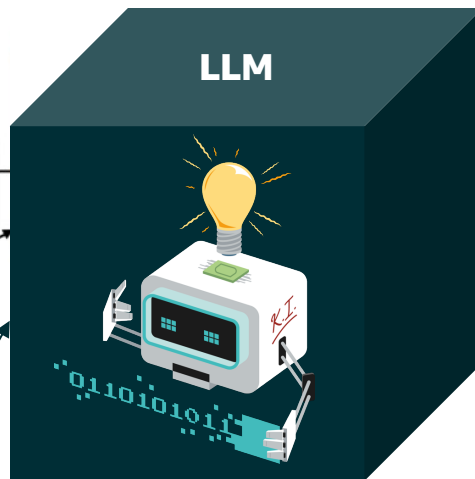
Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?



Decoding

- Greedy Search
- Top-K/Top-P sampling
- Beam Search

.....



Background & Motivation

The best generation may not ranked the highest 😞

Restaurants_1.FindRestaurants(city="Mountain View", cuisine="American", price_range="moderate")

Human: I want to find a **burger joint**.

Assistant: In which city?.

Human: In **Mountain View**.

Assistant: Eureka! restaurant is in Mountain View.

Human: Are there any other restaurants in the **moderate price range**?

"Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)"

"the possible values for "cuisine" include ["Mexican", "Chinese", "Indian", "American", "Italian"]

Beam search results ranked by sequence probability:

1. Restaurants_1.FindRestaurants(price_range="moderate", city="Mountain View") **missing cuisine**
2. Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") **missing price_range**
3. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") **missing price_range**
4. Restaurants_1.FindRestaurants(price_range="moderate") **missing cuisine and city**
5. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View", price_range="moderate")



Background & Motivation

The best generation may not ranked the highest 🙄

Restaurants_1.FindRestaurants(city="Mountain View", cuisine="American", price_range="moderate")

Human: I want to find a **burger joint**.

Assistant: In which city?.

Human: In **Mountain View**.

Assistant: Eureka! restaurant is in Mountain View.

Human: Are there any other restaurants in the **moderate price range**?

"Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)"

"the possible values for "cuisine" include ["Mexican", "Chinese", "Indian", "American", "Italian"]"

Beam search results ranked by sequence probability:

1. Restaurants_1.FindRestaurants(price_range="moderate", city="Mountain View") missing *cuisine*
2. Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") missing *price_range*
3. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") missing *price_range*
4. Restaurants_1.FindRestaurants(price_range="moderate") missing *cuisine* and *city*
5. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View", price_range="moderate")



Background & Motivation

The best generation may not ranked the highest 🙄

Restaurants_1.FindRestaurants(city="Mountain View", cuisine="American", price_range="moderate")

Human: I want to find a **burger joint**.

Assistant: In which city?.

Human: In **Mountain View**.

Assistant: Eureka! restaurant is in Mountain View.

Human: Are there any other restaurants in the **moderate price range**?

"Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)"

"the possible values for "cuisine" include ["Mexican", "Chinese", "Indian", "American", "Italian"]"

Beam search results ranked by sequence probability:

1. Restaurants_1.FindRestaurants(price_range="moderate", city="Mountain View") missing *cuisine*
2. Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") missing *price_range*
3. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") missing *price_range*
4. Restaurants_1.FindRestaurants(price_range="moderate") missing *cuisine* and *city*
5. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View", price_range="moderate")



Background & Motivation

The best generation may not ranked the highest 🙄

Restaurants_1.FindRestaurants(city="Mountain View", cuisine="American", price_range="moderate")

Human: I want to find a burger joint.

Assistant: In which city?.

Human: In Mountain View.

Assistant: Eureka! restaurant is in Mountain View.

Human: Are there any other restaurants in the moderate price range?

"Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)"

"the possible values for "cuisine" include ["Mexican", "Chinese", "Indian", "American", "Italian"]

Beam search results ranked by sequence probability:

1. Restaurants_1.FindRestaurants(price_range="moderate", city="Mountain View") missing *cuisine*
2. Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") missing *price_range*
3. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") missing *price_range*
4. Restaurants_1.FindRestaurants(price_range="moderate") missing *cuisine* and *city*
5. **Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View", price_range="moderate")**



Background & Motivation

The best generation may not ranked the highest 🙄

`Restaurants_1.FindRestaurants(city="Mountain View", cuisine="American", price_range="moderate")`

Human: I want to find a burger joint.

Assistant: In which city?.

Human: In Mountain View.

Assistant: Eureka! restaurant is in Mountain View.

Human: Are there any other restaurants in the moderate price range?

`"Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)"`

`"the possible values for "cuisine" include ["Mexican", "Chinese", "Indian", "American", "Italian"]`

Beam search results ranked by sequence probability:

1. `Restaurants_1.FindRestaurants(price_range="moderate", city="Mountain View")` **missing cuisine**
2. `Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View")` **missing price_range**
3. `Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View")` **missing price_range**
4. `Restaurants_1.FindRestaurants(price_range="moderate")` **missing cuisine and city**
5. `Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View", price_range="moderate")`



Background & Motivation

The best generation may not ranked the highest 🙄

`Restaurants_1.FindRestaurants(city="Mountain View", cuisine="American", price_range="moderate")`

Human: I want to find a **burger joint**.

Assistant: In which city?.

Human: In Mountain View.

Assistant: Eureka! restaurant is in Mountain View.

Human: Are there any other restaurants in the moderate price range?

"Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)"

"the possible values for "cuisine" include ["Mexican", "Chinese", "Indian", "American", "Italian"]

Beam search results ranked by sequence probability:

1. `Restaurants_1.FindRestaurants(price_range="moderate", city="Mountain View")` missing *cuisine*
2. `Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View")` missing *price_range*
3. `Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View")` missing *price_range*
4. `Restaurants_1.FindRestaurants(price_range="moderate")` missing *cuisine* and *city*
5. `Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View", price_range="moderate")`



Background & Motivation

The best generation may not ranked the highest 🙄

Restaurants_1.FindRestaurants(city="Mountain View", cuisine="American", price_range="moderate")

Human: I want to find a **burger joint**.

Assistant: In which city?

Human: In Mountain View.

Assistant: Eureka! restaurant is in Mountain View.

Human: Are there any other restaurants in the moderate price range?

"Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)"

"the possible values for "cuisine" include ["Mexican", "Chinese", "Indian", "American", "Italian"]

Beam search results ranked by sequence probability:

1. Restaurants_1.FindRestaurants(price_range="moderate", city="Mountain View") missing *cuisine*
2. Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") missing *price_range*
3. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") missing *price_range*
4. Restaurants_1.FindRestaurants(price_range="moderate") missing *cuisine* and *city*
5. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View", price_range="moderate")



Background & Motivation

Quantitative Analysis of the ICL-LLM's Generation

Data: 100 samples from DSTC8

Model: Alpaca-13B

Setting:

Prompt the LLM with *documentation of short-listed APIs* and *exemplars of short-listed APIs* in addition to the conversation history

Decoding Algorithm:

Beam search with beam size as 10.

Evaluation:

Code Element Match Score - API calls are generated with correct arguments and correct argument values

Restaurants_1.FindRestaurants(city="Mountain View", cuisine="American", price_range="moderate")



Restaurants_1

FindRestaurants

city="Mountain View"

cuisine="American"

price_range="moderate"

(symmetric difference)

$$\frac{|A \setminus B| + |B \setminus A|}{|A| + |B|}$$

(total elements amount)

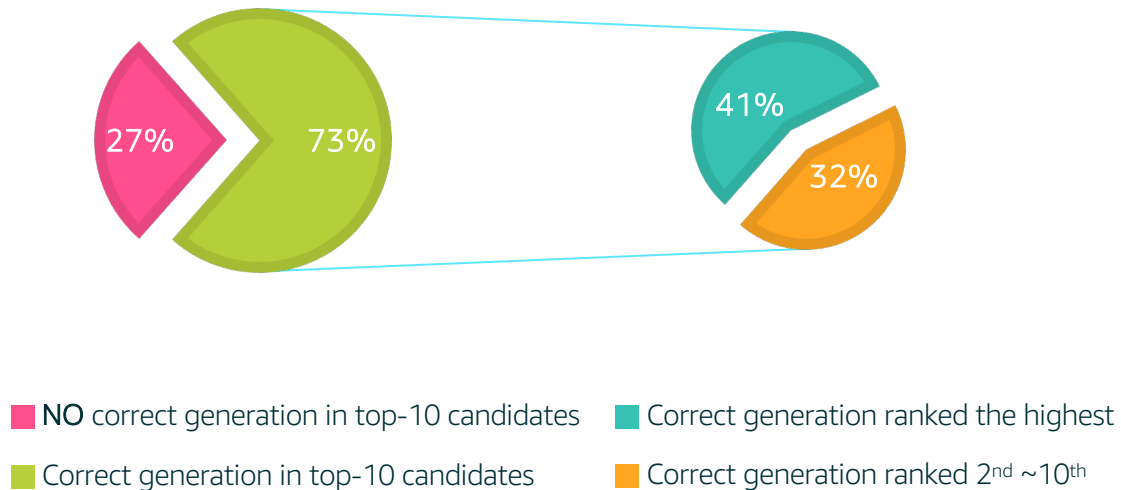
where A is the elements of candidate generation and B is the elements of the reference.

A correct generation is the case in which the CEMS is 1.

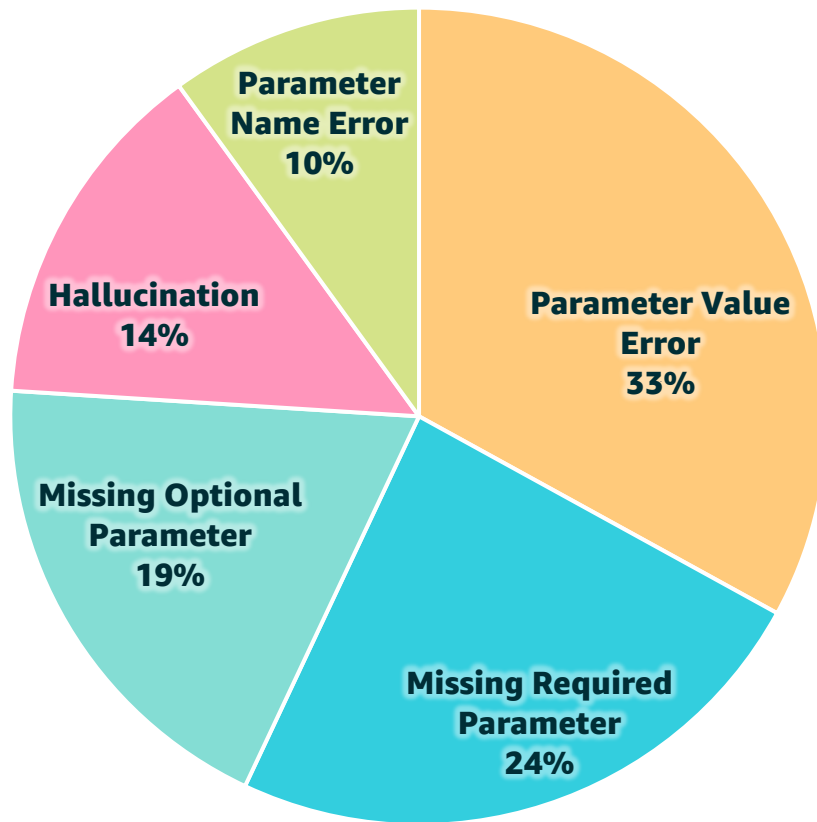


Background & Motivation

Quantitative Analysis

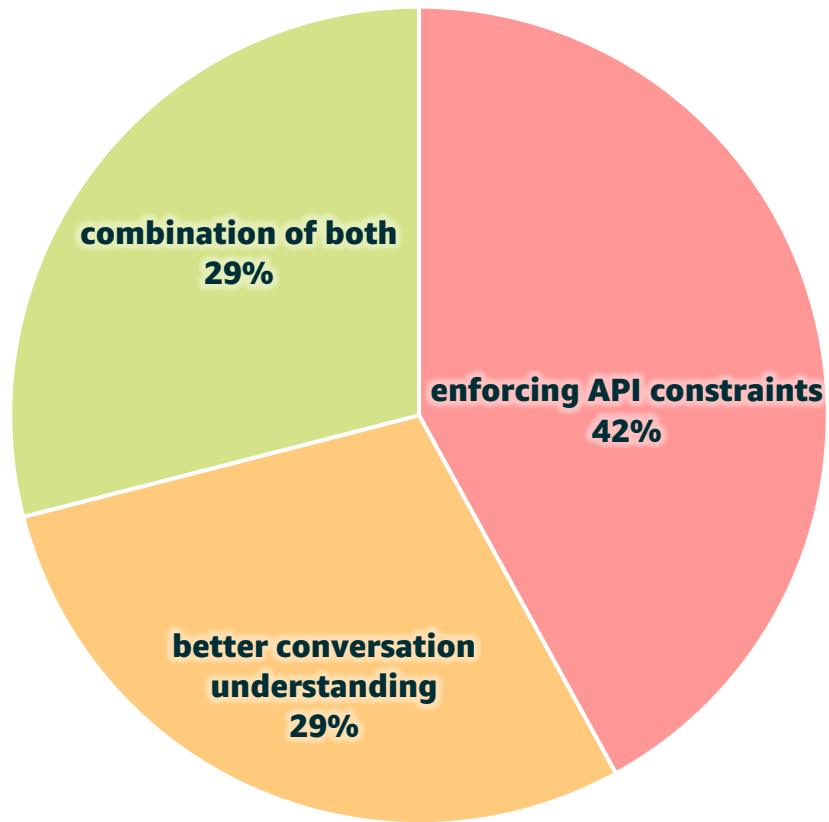


Qualitative Analysis – Error Types



Background & Motivation

Qualitative Analysis – Fixing the Error



Background & Motivation

Fantastic sequence and where to find it 🤔

Restaurants_1.FindRestaurants(city="Mountain View", cuisine="American", price_range="moderate")

1. Restaurants_1.FindRestaurants(price_range="moderate", city="Mountain View") *missing cuisine*
2. Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") *missing price_range*
3. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") *missing price_range*
4. Restaurants_1.FindRestaurants(price_range="moderate") *missing cuisine and city*
5. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View", price_range="moderate")



Proposed Method

Fantastic sequence and where to find it 🤗

Restaurants_1.FindRestaurants(city="Mountain View", cuisine="American", price_range="moderate")

1. Restaurants_1.FindRestaurants(price_range="moderate", city="Mountain View") **missing cuisine**
2. Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") **missing price_range**
3. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") **missing price_range**
4. Restaurants_1.FindRestaurants(price_range="moderate") **missing cuisine and city**
5. **Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View", price_range="moderate")**

Our Solution:

Reranking and Constrained Decoding!

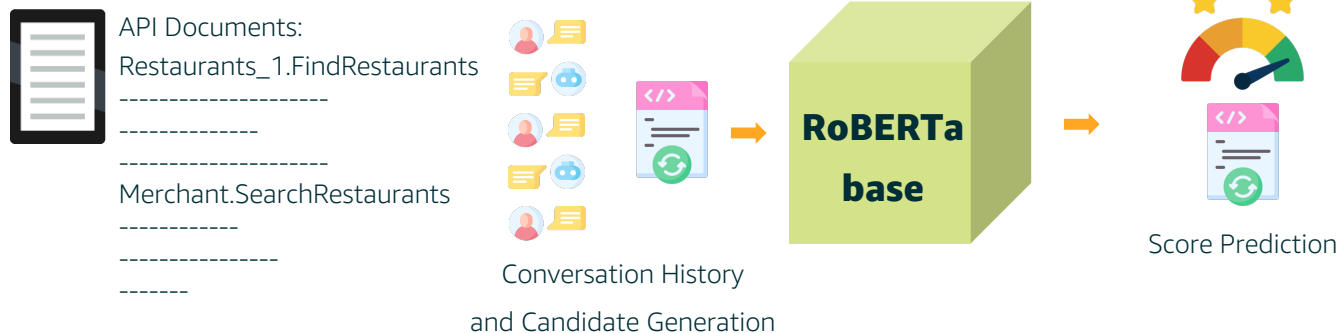


RQ1: Can small supervised model effectively discriminate the generation of LLM?



Proposed Method

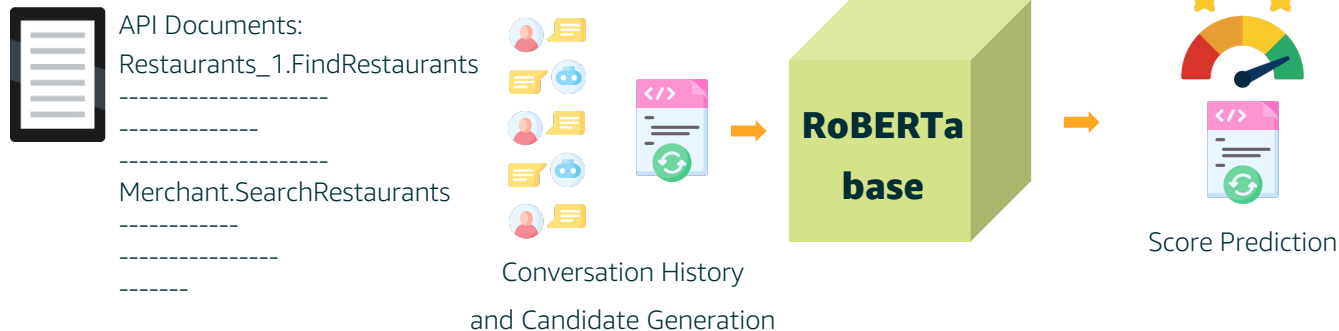
Reranking - Training of the Scorer



- Model: RoBERTa-base (**125M** parameters)
- Data: **1,800 / 200** training / testing samples
 - + associated candidate generations for each sample
- Trained with **MSE Loss + Spearman (Soft) Ranking Correlation Loss**
- MSE: 0.020074 / **RMSE: 0.1416**

Proposed Method

Reranking - Training of the Scorer



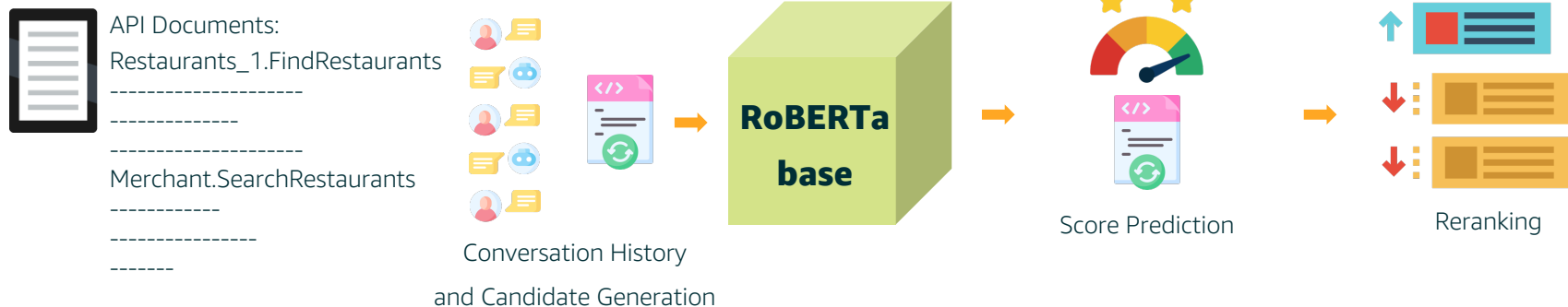
Restaurants_1.FindRestaurants(city="Mountain View", cuisine="American", price_range="moderate")

1. Restaurants_1.FindRestaurants(price_range="moderate", city="Mountain View") *missing cuisine* 0.8
2. Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") *missing price_range* 0.6
3. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") *missing price_range* 0.8
4. Restaurants_1.FindRestaurants(price_range="moderate") *missing cuisine and city* 0.6
5. Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View", price_range="moderate") 1.0



Proposed Method

Reranking with the Predicted Score

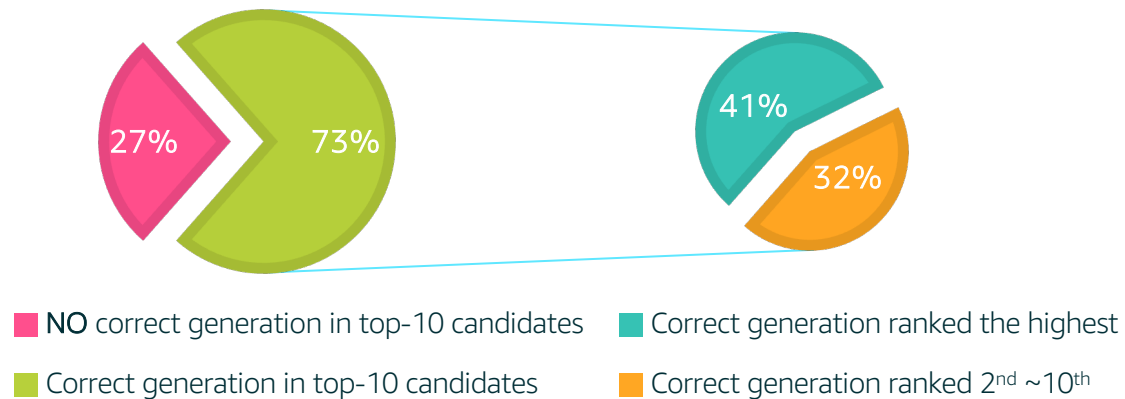


Restaurants_1.FindRestaurants(city="Mountain View", cuisine="American", price_range="moderate")

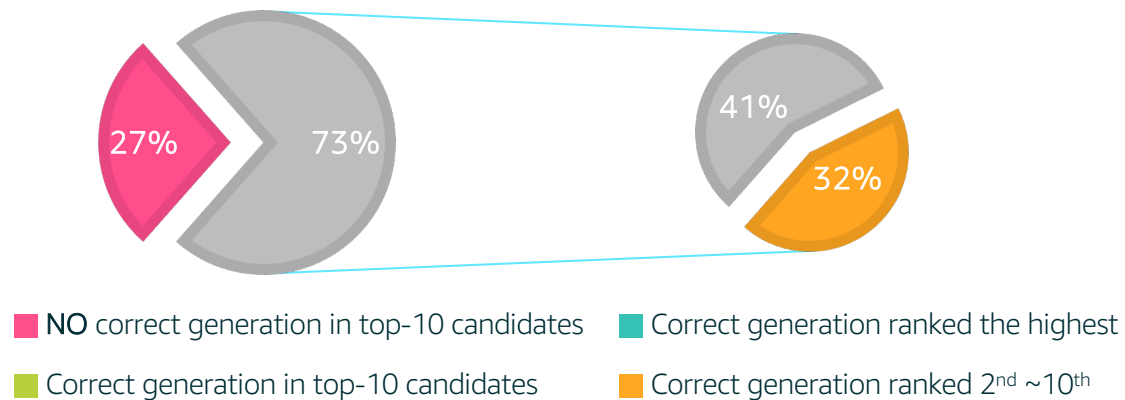
1. **Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View", price_range="moderate")** 1.0
2. **Restaurants_1.FindRestaurants(price_range="moderate", city="Mountain View")** missing *cuisine* 0.8
3. **Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View")** missing *price_range* 0.8
4. **Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View")** missing *price_range* 0.6
5. **Restaurants_1.FindRestaurants(price_range="moderate")** missing *cuisine* and *city* 0.6



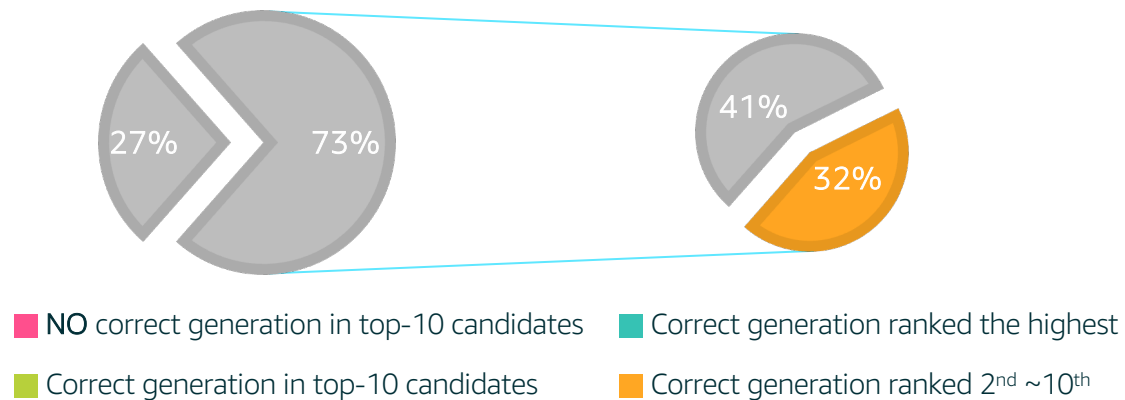
Proposed Method: Reranking Issue with Reranking



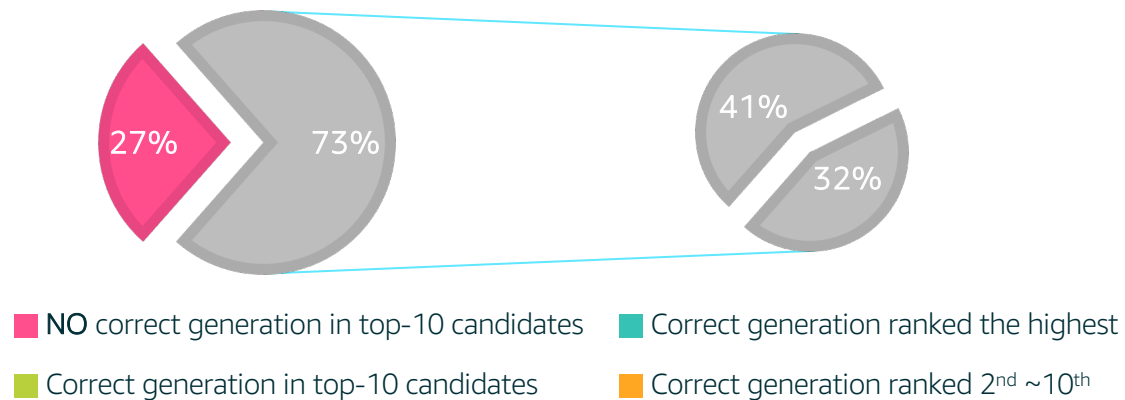
Proposed Method: Reranking Issue with Reranking



Proposed Method: Reranking Issue with Reranking



Proposed Method: Reranking Issue with Reranking



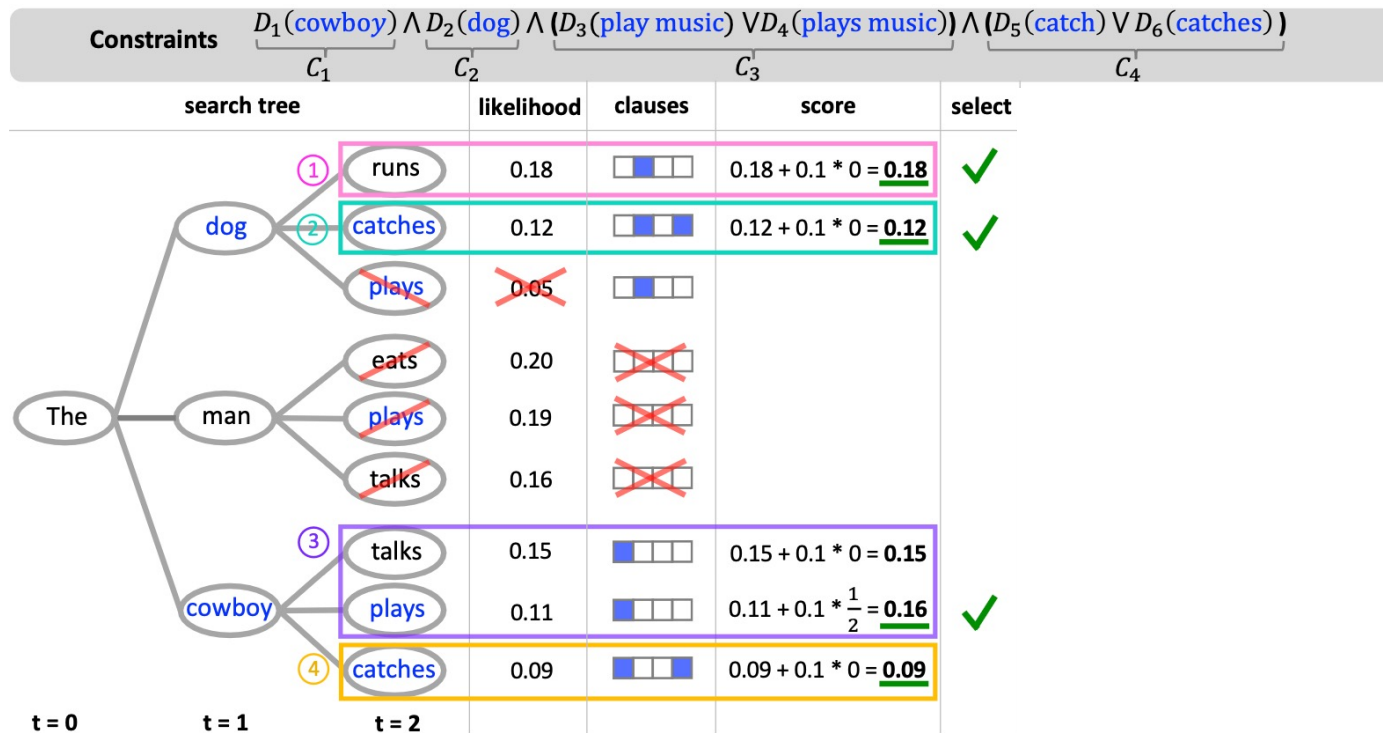
**RQ2: How to intervene the generation
before the full sequence is decoded?**



RQ3: Can the API generation related constraints be enforced without the reliance on labeled data and additional models?

Related Work

Conventional Constrained Decoding



Related Work

Conventional Constrained Decoding

Constraints						
$D_1(\text{cowboy}) \wedge D_2(\text{dog}) \wedge (D_3(\text{play music}) \vee D_4(\text{plays music})) \wedge (D_5(\text{catch}) \vee D_6(\text{catches}))$						
		C_1	C_2	C_3	C_4	
search tree	likelihood	clauses	score	select		
The dog man cowboy	①	runs	0.18		$0.18 + 0.1 * 0 = \underline{0.18}$	✓
	②	catches	0.12		$0.12 + 0.1 * 0 = \underline{0.12}$	✓
		plays	0.05			
		eats	0.20			
		plays	0.19			
		talks	0.16			
	③	talks	0.15		$0.15 + 0.1 * 0 = \underline{0.15}$	
		plays	0.11		$0.11 + 0.1 * \frac{1}{2} = \underline{0.16}$	✓
	④	catches	0.09		$0.09 + 0.1 * 0 = \underline{0.09}$	
t = 0	t = 1	t = 2				

- Unable to handle
and one-to-many

Token occurrence con
parameter1 & param
parameter2-values

parameter1=paramet
parameter2= paramet

- Unable to handle the one-to-one and one-to-many relations

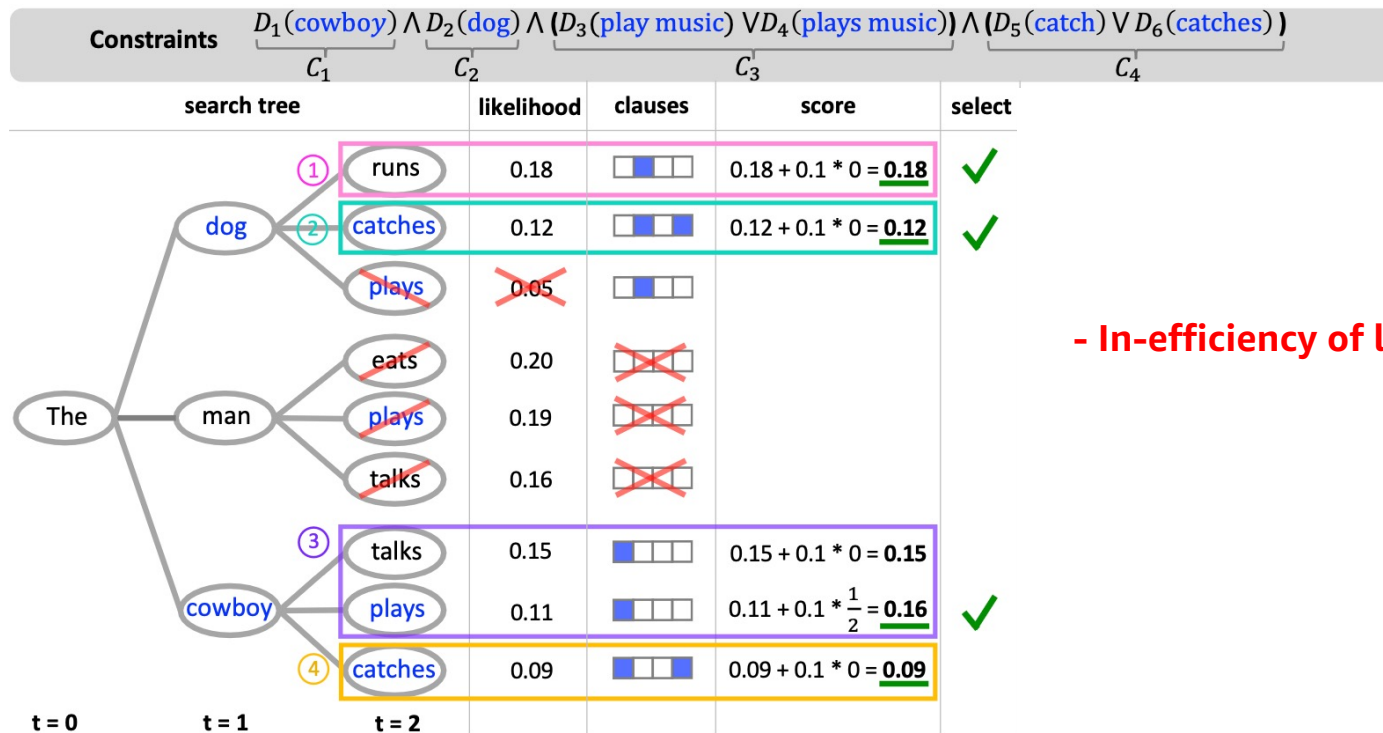
Token occurrence constraints:
parameter1 & parameter2 & parameter1-values & parameter2-values

parameter1=parameter2-values,
parameter2= parameter1-values



Related Work

Conventional Constrained Decoding



- In-efficiency of look ahead decoding



Proposed Method: State-tracked Constrained Decoding For a Conventional Decoding Step...

Context

 I want to find a **burger joint**.

In which city?

 In **Mountain View**.

Eureka! restaurant is in
Mountain View.

 Are there any other
restaurants in the **moderate
price range**?



Proposed Method: State-tracked Constrained Decoding For a Conventional Decoding Step...

Context

 I want to find a **burger joint**.

In which city?

 In **Mountain View**.

Eureka! restaurant is in
Mountain View.

 Are there any other
restaurants in the **moderate
price range**?



API Documents:

`Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)`

the possible values for "cuisine" include ["Mexican", "Chinese", "Indian", "American", "Italian"]



Proposed Method: State-tracked Constrained Decoding For a Conventional Decoding Step...

Context

 I want to find a **burger joint**.

In which city?

 In **Mountain View**.

Eureka! restaurant is in
Mountain View.

 Are there any other
restaurants in the **moderate
price range**?

Restaurants_1.FindRestaurants(cuisine="



API Documents:

Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)

the possible values for "cuisine" include ["Mexican", "Chinese", "Indian", "American", "Italian"]



Proposed Method: State-tracked Constrained Decoding For a Conventional Decoding Step...

Context

 I want to find a **burger joint**.

In which city?

 In **Mountain View**.

Eureka! restaurant is in
Mountain View.

 Are there any other
restaurants in the **moderate
price range**?

Restaurants_1.FindRestaurants(cuisine="



API Documents:

Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)

the possible values for "cuisine" include ["Mexican", "Chinese", "Indian", "American", "Italian"]

American	.3
Burger	.4
Chinese	.2
Delicious	.05
(others)	



Proposed Method: State-tracked Constrained Decoding For a Conventional Decoding Step...

Context

 I want to find a **burger joint**.

In which city?

 In **Mountain View**.

Eureka! restaurant is in
Mountain View.

 Are there any other
restaurants in the **moderate
price range**?

Restaurants_1.FindRestaurants(cuisine="



American	.3
Burger	.4
Chinese	.2
Delicious	.05
(others)	

API Documents:

Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)

the possible values for "cuisine" include

["Mexican", "Chinese", "Indian", "American", "Italian"]

Proposed Method

State-tracked Constrained Decoding

Context

-  I want to find a **burger joint**.
In which city? 
-  In **Mountain View**.
Eureka! restaurant is in Mountain View. 
-  Are there any other restaurants in the **moderate price range**?

Restaurants_1.FindRestaurants(cuisine="



Track the State of the Generation
The model is about to generate the value of cuisine !



API Documents:

Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)

the possible values for "cuisine" include

["Mexican", "Chinese", "Indian", "American", "Italian"]



Proposed Method

State-tracked Constrained Decoding

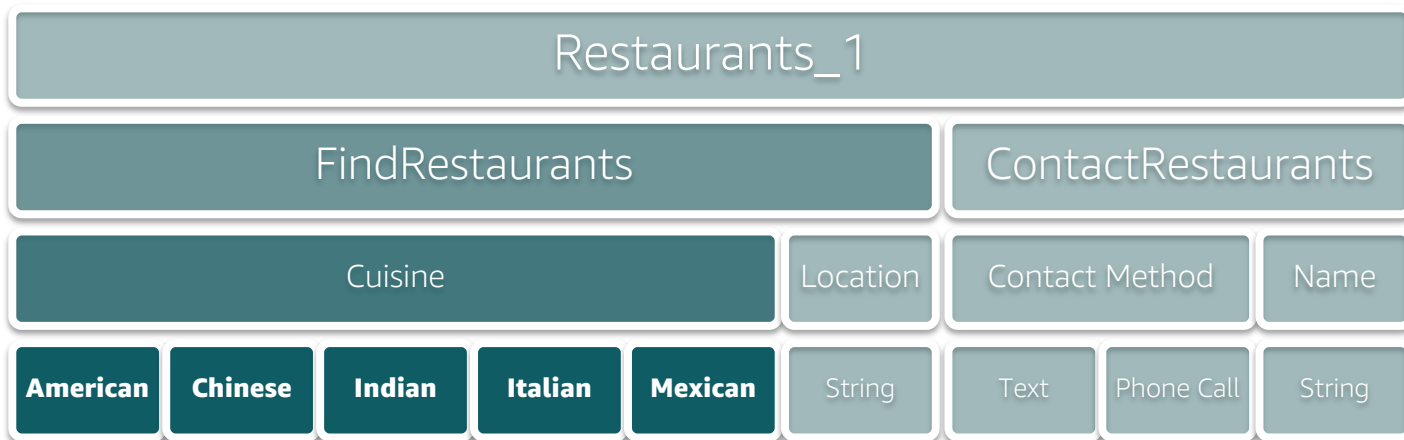


API Documents:

Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)

the possible values for "cuisine" include

["Mexican", "Chinese", "Indian", "American", "Italian"]



Proposed Method: State-tracked Constrained Decoding

Recall the Conventional Decoding Step...

Context

 I want to find a **burger joint**.

In which city?

 In **Mountain View**.

Eureka! restaurant is in
Mountain View.

 Are there any other
restaurants in the **moderate
price range**?

Restaurants_1.FindRestaurants(cuisine="



American	.3
Burger	.4
Chinese	.2
Delicious	.05
(others)	

API Documents:

Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)

the possible values for "cuisine" include

["Mexican", "Chinese", "Indian", "American", "Italian"]

Proposed Method

State-tracked Constrained Decoding

Context

 I want to find a **burger joint**.

In which city?

 In **Mountain View**.

Eureka! restaurant is in
Mountain View.

 Are there any other
restaurants in the **moderate
price range**?

Restaurants_1.FindRestaurants(cuisine="



API Documents:

Restaurants_1.FindRestaurants("cuisine" : Required, "city" : Required, "price_range" : Optional, "has_live_music" : Optional, "serves_alcohol" : Optional)

the possible values for "cuisine" include

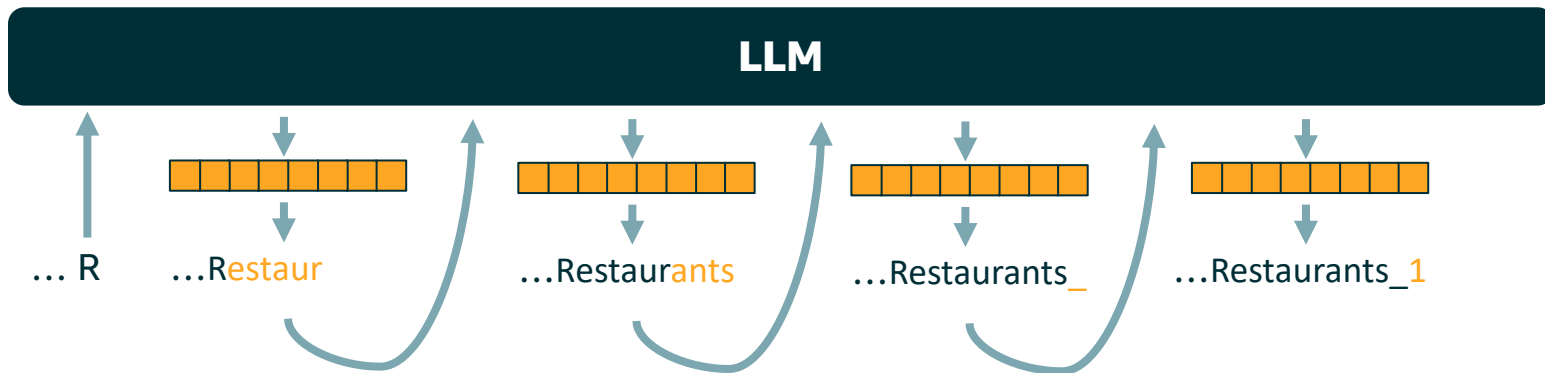
["Mexican", "Chinese", "Indian", "American", "Italian"]

	American	.3
	Chinese	.2
	Indian	.01
	Italian	.01
	Mexican	.01

Proposed Method: State-tracked Constrained Decoding

Compute Efficiency

1. R
 2. Restaur
 3. Restaurants
 4. Restaurants_
 5. Restaurants_1
- » Tokenizers of LLMs perform WordPiece tokenization
- » LLMs generate autoregressively Piece-by-Piece



Proposed Method: State-tracked Constrained Decoding

Compute Efficiency

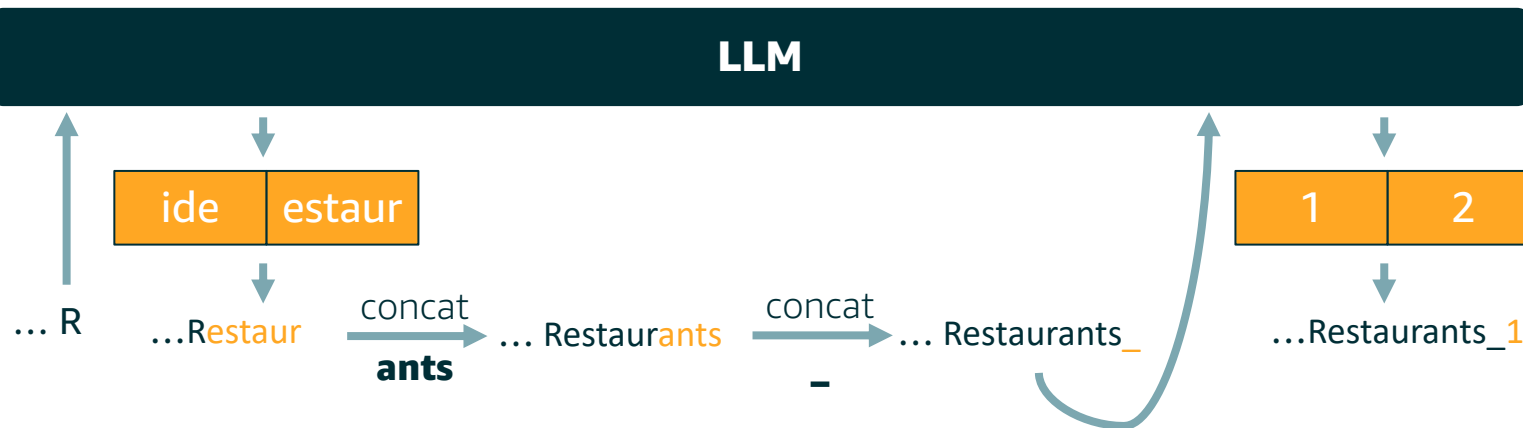
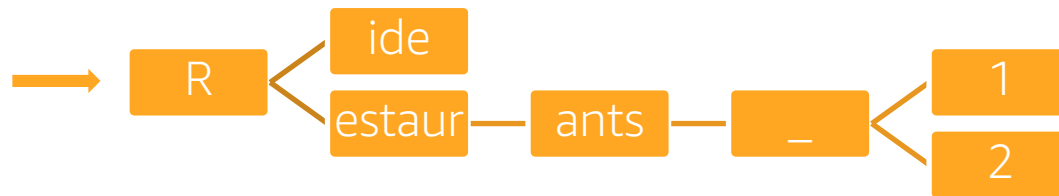


API Documents:

Restaurants_1.FindRestaurants

Restaurants_2.SearchRestaurants

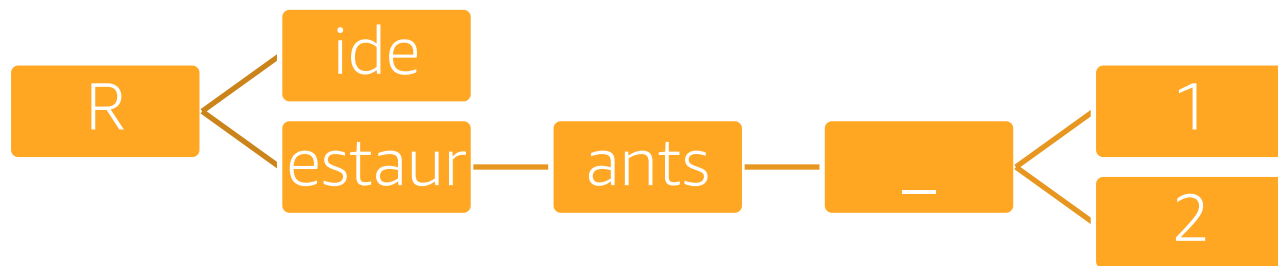
Ride.FindAvailableRide



Proposed Method: State-tracked Constrained Decoding

Compute Efficiency

1. R
2. Restaur
3. Restaurants
4. Restaurants_
5. Restaurants_1



Normal Decoding	Inference	Inference	Inference	Inference
State-tracked Constrained Decoding	Inference	<u>Concat</u>	<u>Concat</u>	Inference



Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps

Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") *missing price_range*

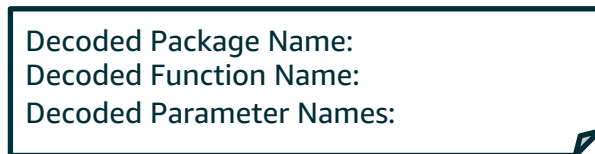
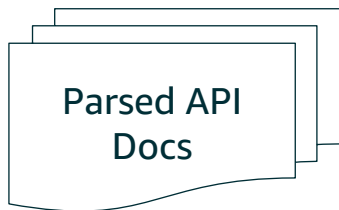
- User want's to find a *burger joint* at *Mountain View* within the *moderate price range*
- **Wrong** *cuisine* value and **missed** the required parameter of *price_range*



Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps

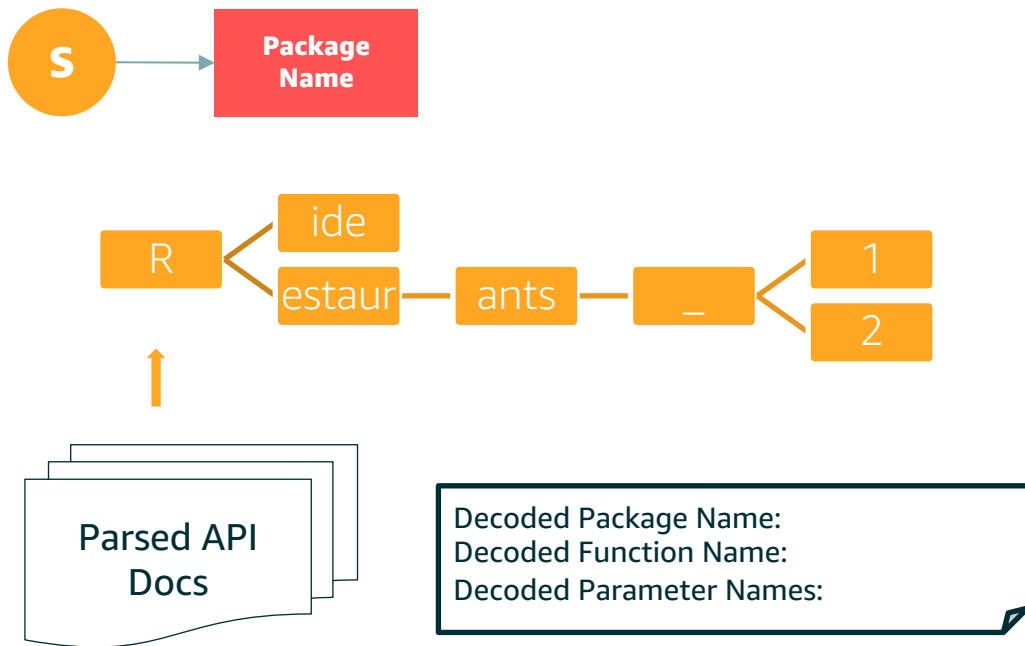
Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") missing *price_range*



Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps

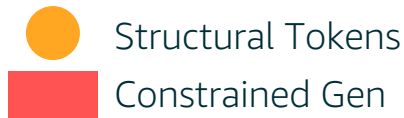
Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") missing *price_range*



- Fetch the constrained *package name* token-search-trie
- Perform constrained decoding

Proposed Method: State-tracked Constrained Decoding

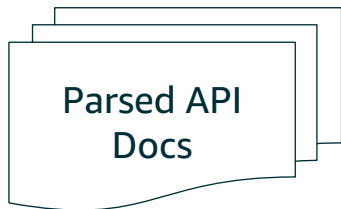
Illustration of Our Decoding Steps



Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") missing *price_range*



- The structural DOT token
- Record decoded package name

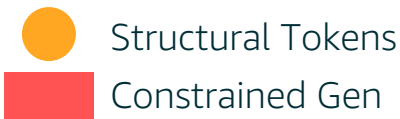


Decoded Package Name: Restaurants_1
Decoded Function Name:
Decoded Parameter Names:

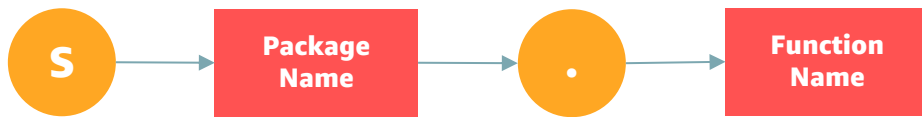


Proposed Method: State-tracked Constrained Decoding

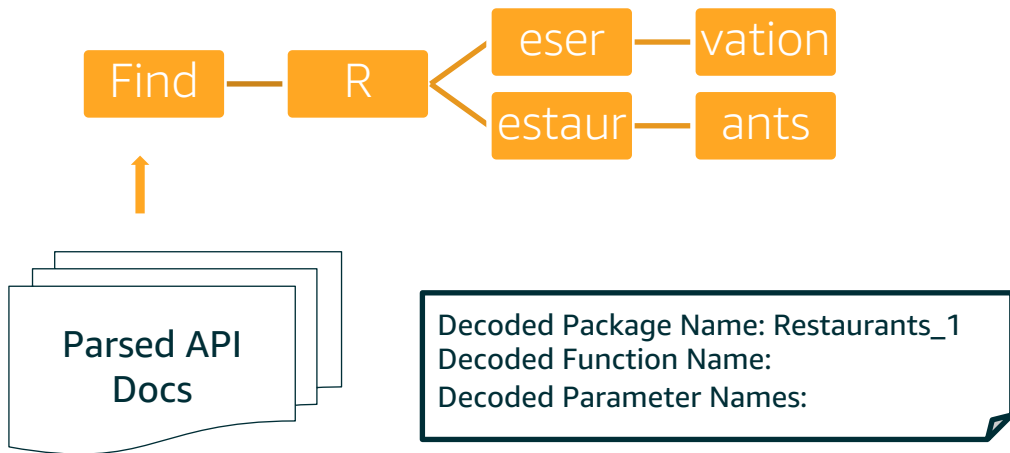
Illustration of Our Decoding Steps



Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") missing *price_range*



- Use *package name* as query key
- Fetch the constrained *function name* token-search-trie
- Perform constrained decoding

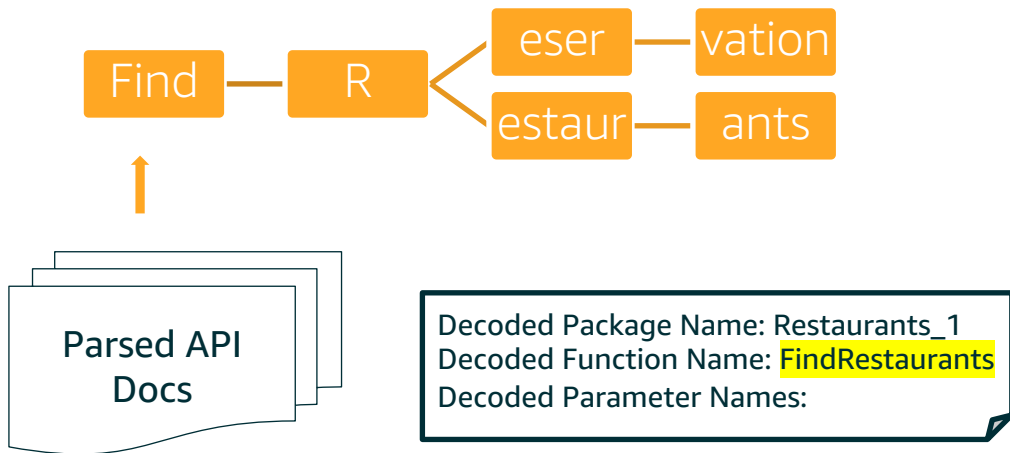


Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps



Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") missing *price_range*

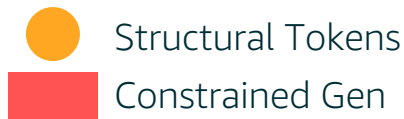


- The structural LEFT_BRACKET token
- Record decoded function name

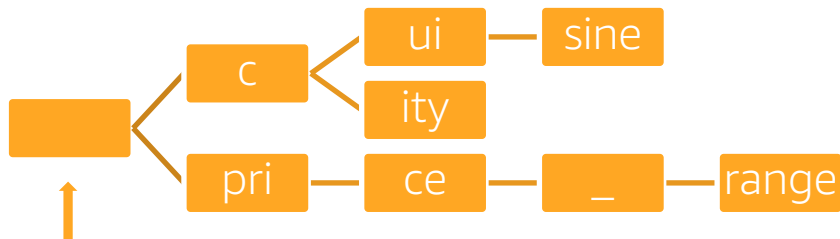


Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps



Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") missing *price_range*



- Use *package name* and *function name* as query keys
- Fetch the constrained *parameter name* token-search-trie
- Perform constrained decoding

Parsed API
Docs

Decoded Package Name: Restaurants_1
Decoded Function Name: FindRestaurants
Decoded Parameter Names:

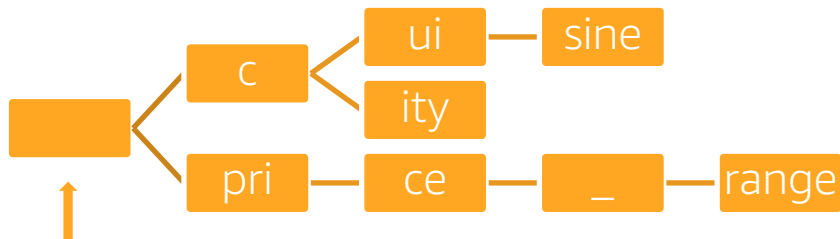
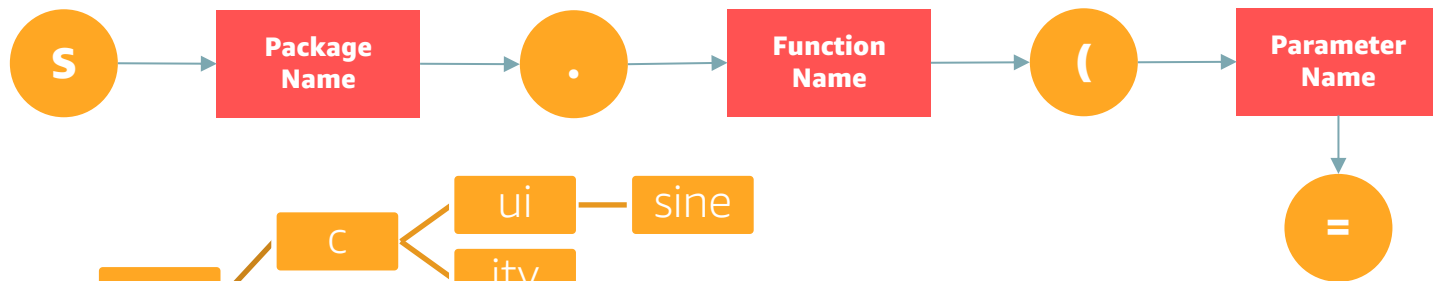


Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps



Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") missing *price_range*



- The structural EQUAL token
- Record decoded parameter name

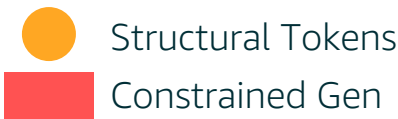
Parsed API
Docs

Decoded Package Name: Restaurants_1
Decoded Function Name: FindRestaurants
Decoded Parameter Names: [cuisine]

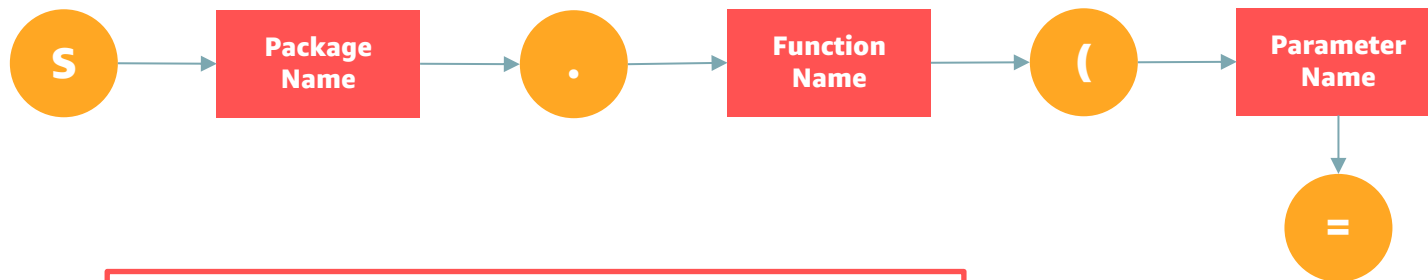


Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps



Restaurants_1.FindRestaurants(cuisine="Burgers", city="Mountain View") missing *price_range*



Cuisine : Possible Values : (constrained token-search-trie of American, Chinese, India, ...)

Parsed API
Docs

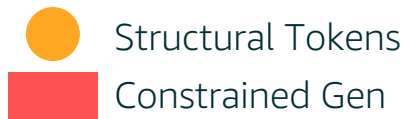
Decoded Package Name: Restaurants_1
Decoded Function Name: FindRestaurants
Decoded Parameter Names: [cuisine]

- Use *package name*, *function name*, and *parameter name* as query keys
- Check if the parameter has a predefined list of possible values

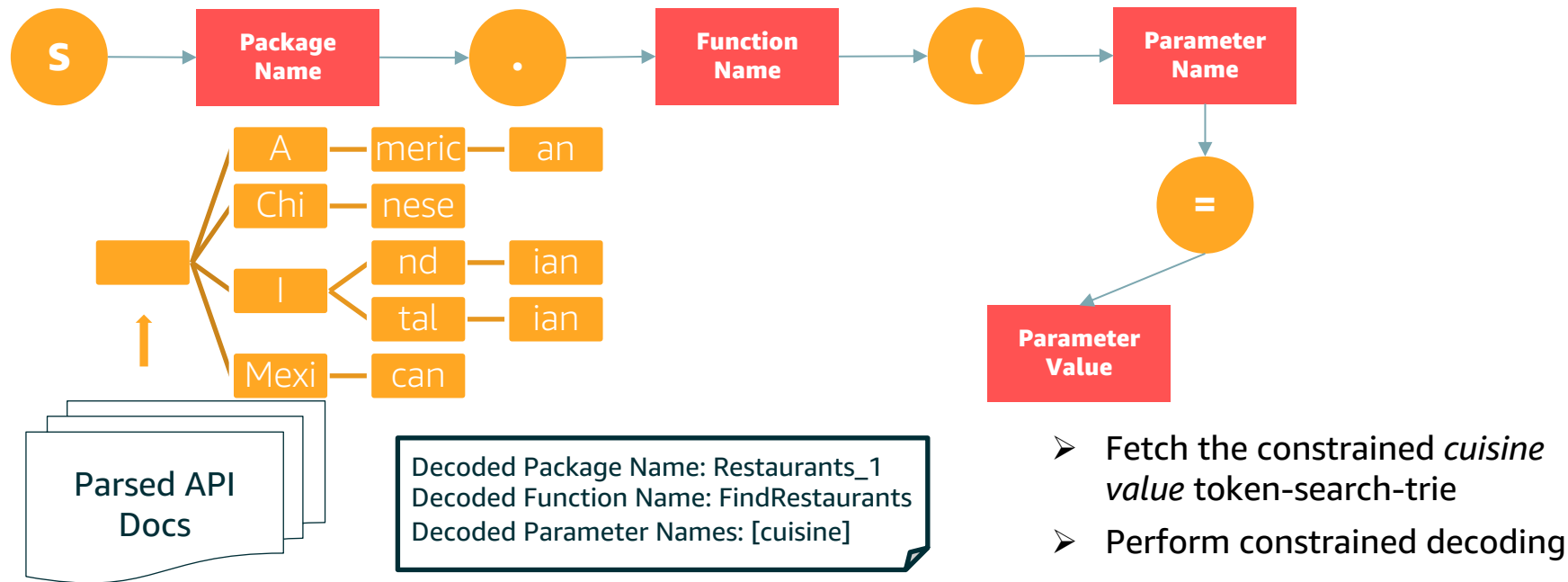


Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps



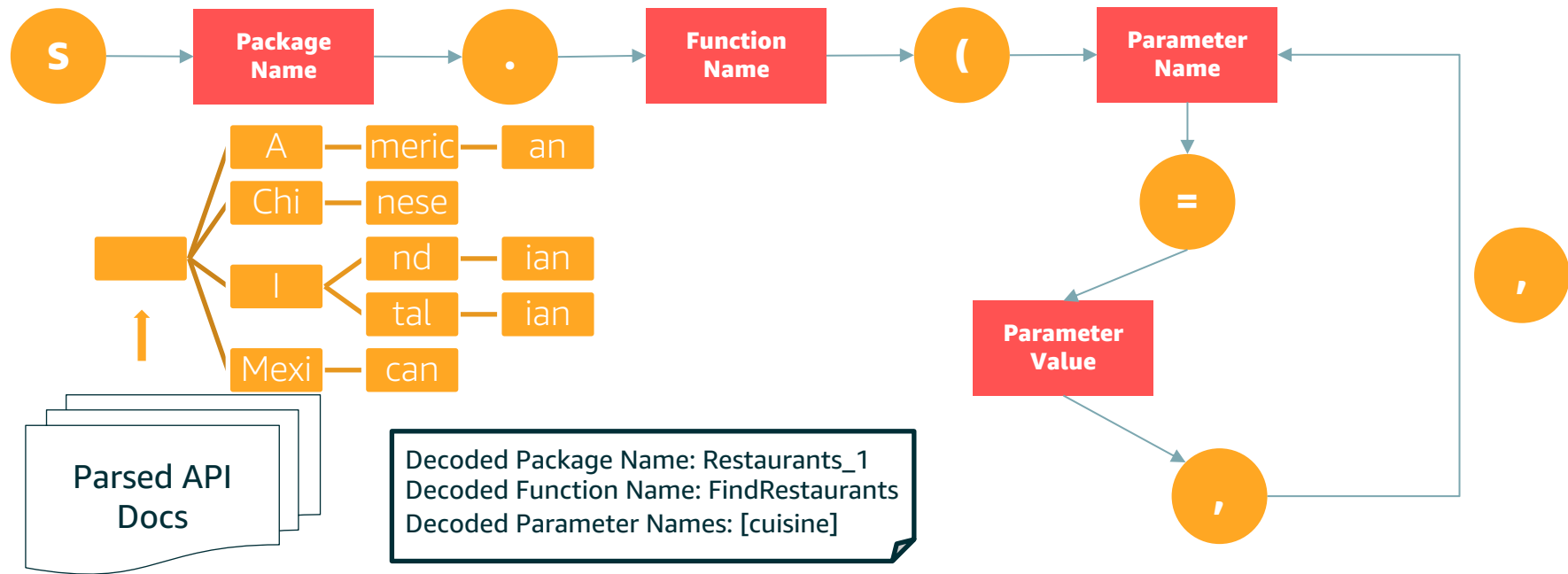
Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") missing *price_range*



Proposed Method: State-tracked Constrained Decoding Illustration of Our Decoding Steps



Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") missing *price_range*

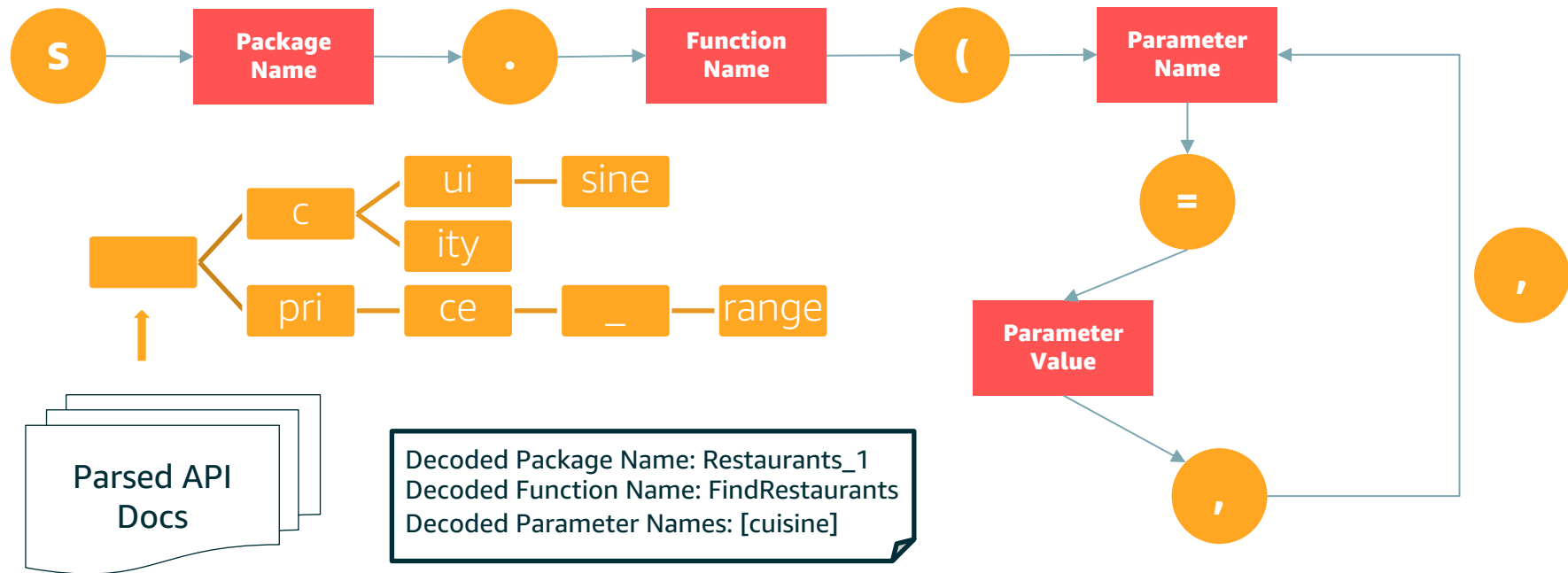


Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps



Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") missing *price_range*

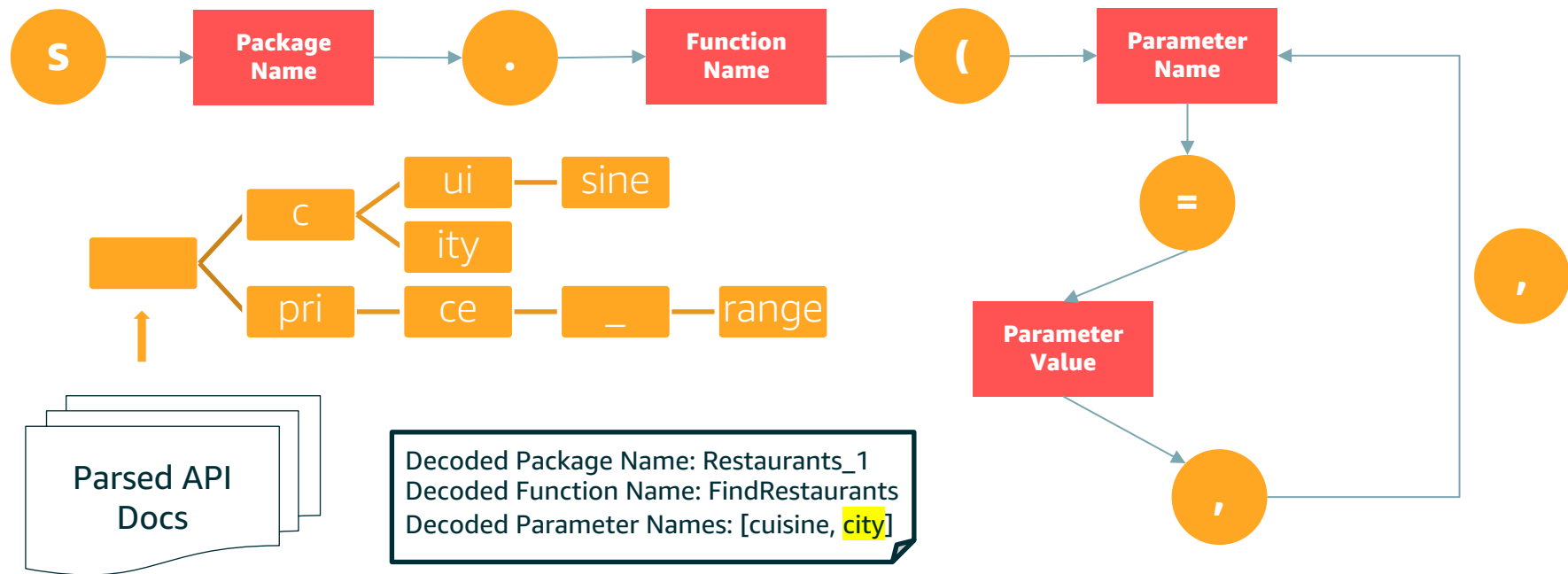


Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps

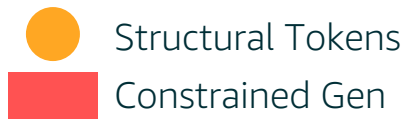


Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") missing *price_range*

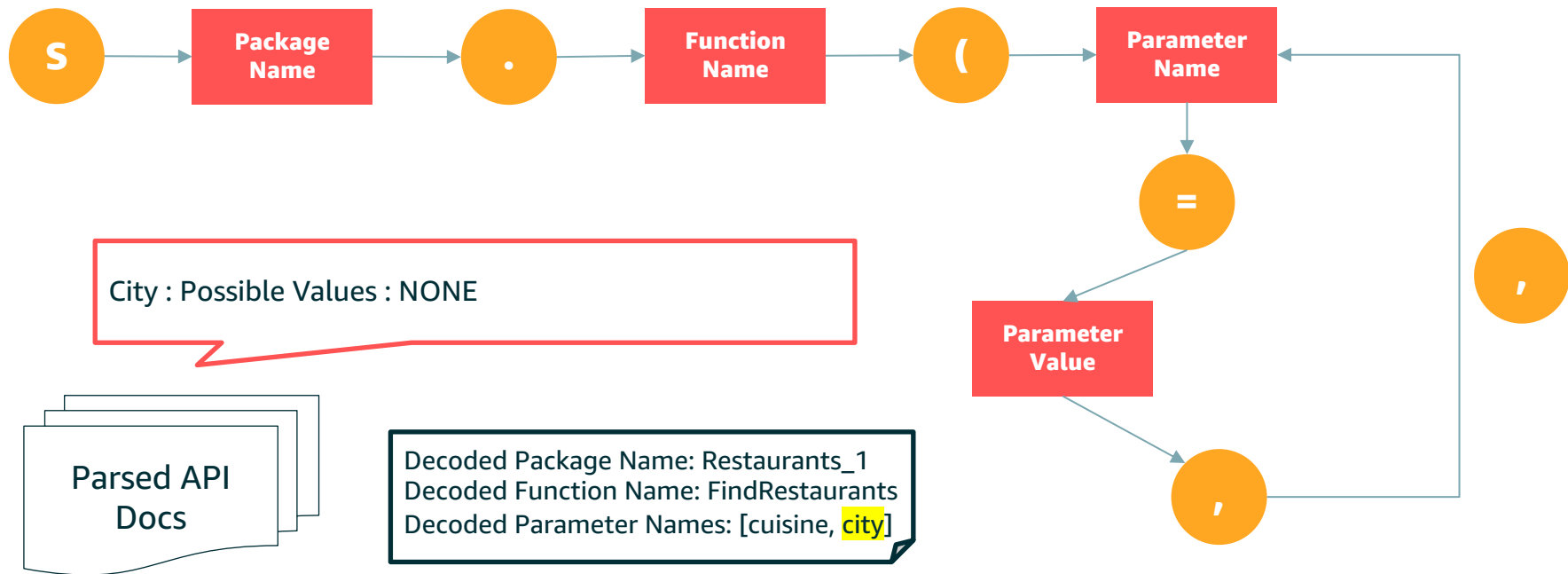


Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps

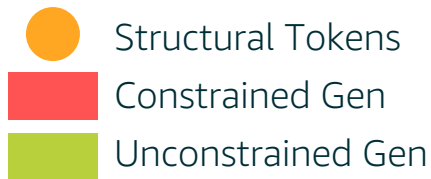


Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") missing *price_range*

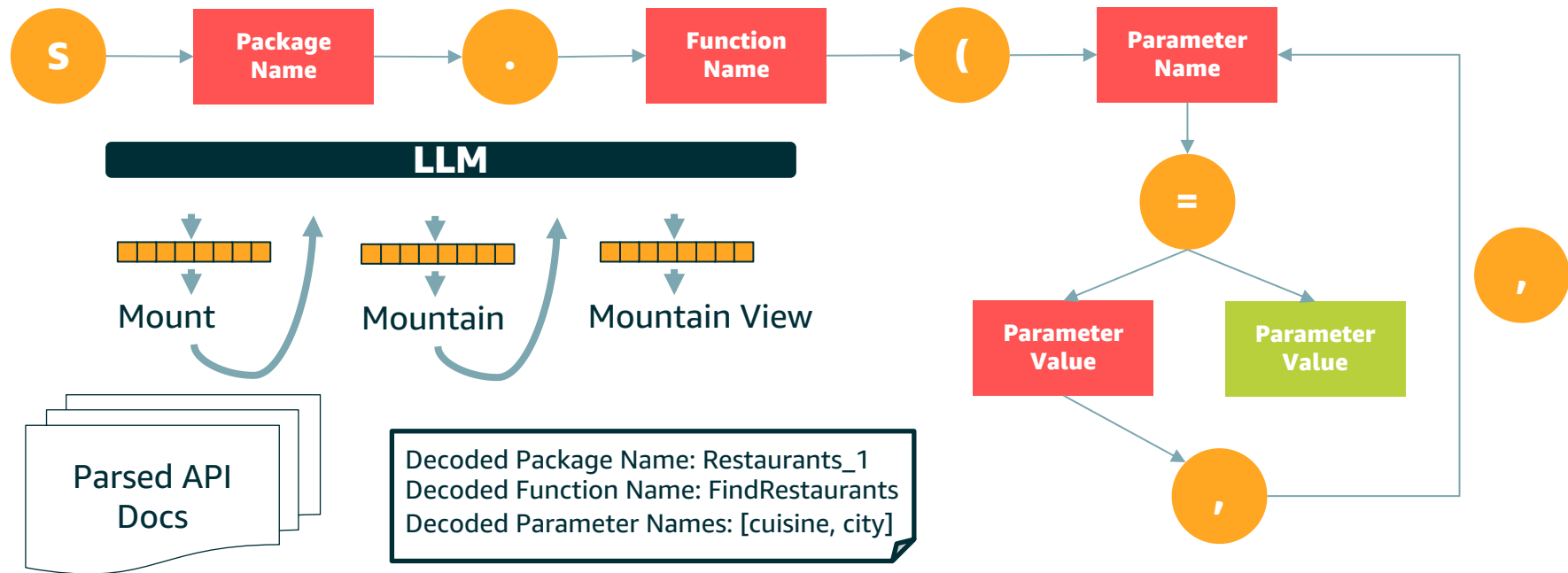


Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps

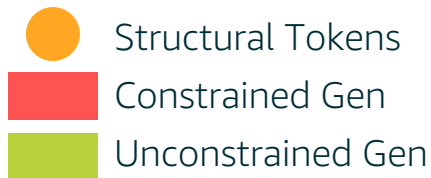


Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") *missing price_range*

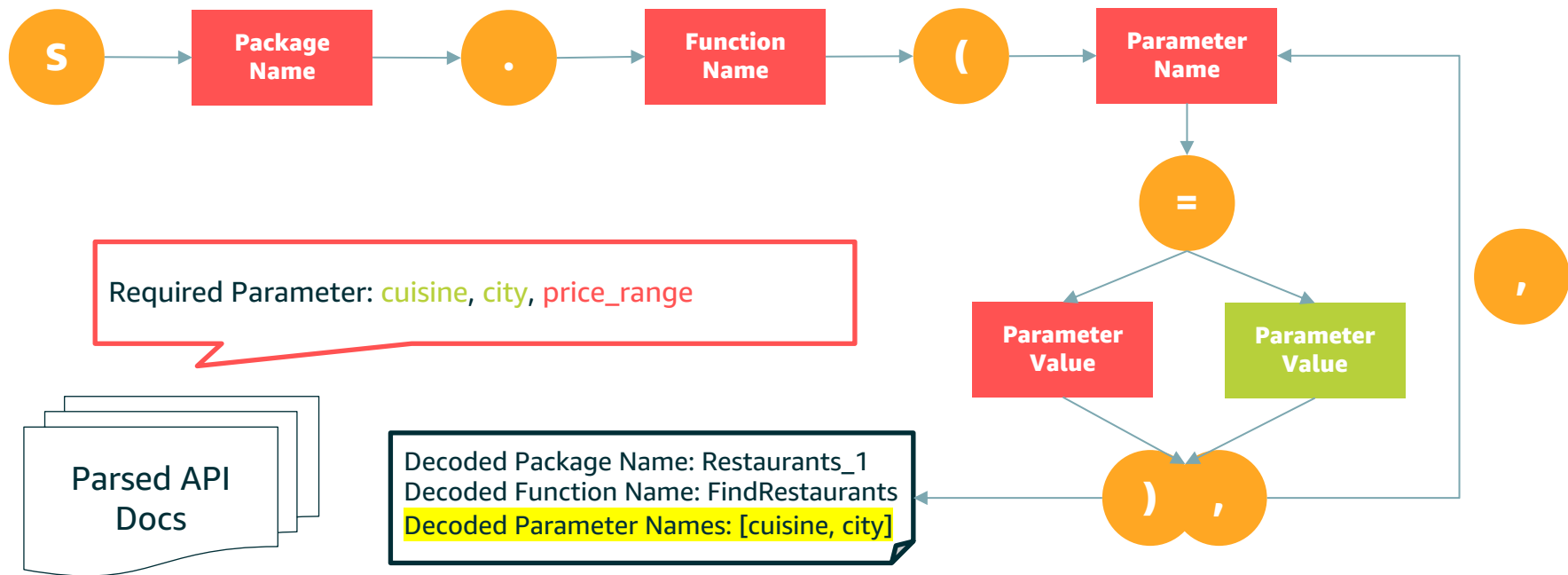


Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps

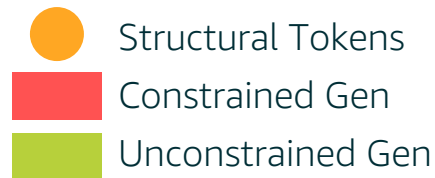


Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View") *missing price_range*

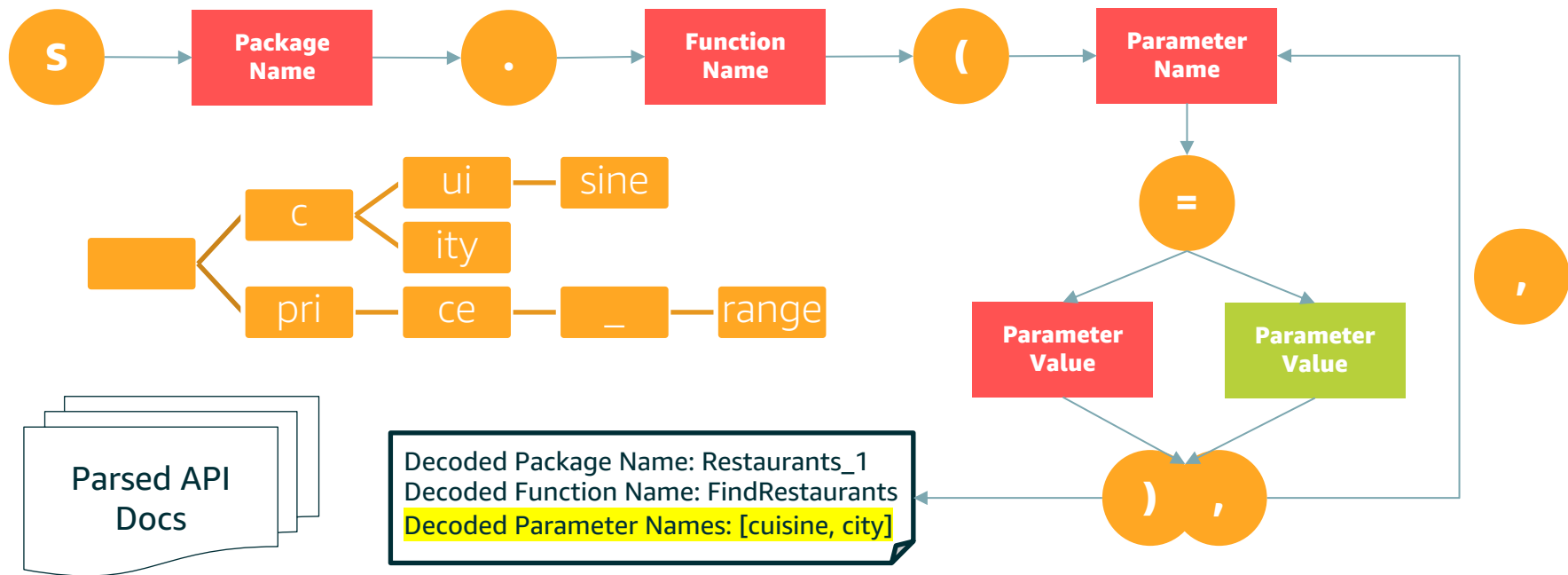


Proposed Method: State-tracked Constrained Decoding

Illustration of Our Decoding Steps



Restaurants_1.FindRestaurants(cuisine="American", city="Mountain View", price_range="moderate")



Experiment Results on DSTC8 and API Bank

DSTC8	API Bank
API Call Generation Task	
Multi-turn Dialogue	
Short-listed APIs and associated Documentaion	
Few-shot In-context Learning with 2 exemplars for each sample	Zero-shot In-context Learning
Settings	
LLM: Alpaca-13B SFT-ed LLM: AlpDSTC-7B	LLM: Alpaca-13B SFT-ed LLM: Lynx-7B
Unconstrained Decoding State-tracked Constrained Decoding	



Experiment Results – Generation Accuracy

Datasets	DSTC8	API Bank
<u>Baseline Methods</u>		
GPT4	51.33	63.66
GPT3.5-turbo	49.28	59.40
Alpaca-13B	40.49	24.31
AlpDSTC/Lynx-7B	47.44	50.53
<u>FANTASE</u>		
Alpaca-13B + Reranking	46.42	33.33
Alpaca-13B + SCD	44.17	62.66
Alpaca-13B + SCD + Reranking	48.88	64.41
AlpDSTC/Lynx-7B + SCD	62.78	67.17



Experiment Results – Generation Accuracy

Datasets	DSTC8	API Bank
<u>Basedline Methods</u>		
GPT4	51.33	63.66
GPT3.5-turbo	49.28	59.40
Alpaca-13B	40.49	24.31
AlpDSTC/Lynx-7B	47.44	50.53
<u>FANTASE</u>		
Alpaca-13B + Reranking	46.42	33.33
Alpaca-13B + SCD	44.17	62.66
Alpaca-13B + SCD + Reranking	48.88	64.41
AlpDSTC/Lynx-7B + SCD	62.78	67.17



Experiment Results – Generation Accuracy

Datasets	DSTC8	API Bank
<u>Basedline Methods</u>		
GPT4	51.33	63.66
GPT3.5-turbo	49.28	59.40
Alpaca-13B	40.49	24.31
AlpDSTC/Lynx-7B	47.44	50.53
<u>FANTASE</u>		
Alpaca-13B + Reranking	46.42	33.33
Alpaca-13B + SCD	44.17	62.66
Alpaca-13B + SCD + Reranking	48.88	64.41
AlpDSTC/Lynx-7B + SCD	62.78	67.17



Experiment Results – Generation Accuracy

Datasets	DSTC8	API Bank
<u>Basedline Methods</u>		
GPT4	51.33	63.66
GPT3.5-turbo	49.28	59.40
Alpaca-13B	40.49	24.31
AlpDSTC/Lynx-7B	47.44	50.53
<u>FANTASE</u>		
Alpaca-13B + Reranking	46.42	33.33
Alpaca-13B + SCD	44.17	62.66
Alpaca-13B + SCD + Reranking	48.88	64.41
AlpDSTC/Lynx-7B + SCD	62.78	67.17



Experiment Results – Generation Accuracy

Datasets	DSTC8	API Bank
<u>Baseline Methods</u>		
GPT4	51.33	63.66
GPT3.5-turbo	49.28	59.40
Alpaca-13B	40.49	24.31
AlpDSTC/Lynx-7B	47.44	50.53
<u>FANTASE</u>		
Alpaca-13B + Reranking	46.42	33.33
Alpaca-13B + SCD	44.17	62.66
Alpaca-13B + SCD + Reranking	48.88	64.41
AlpDSTC/Lynx-7B + SCD	62.78	67.17



Experiment Results – Inference Efficiency

Decoding Strategy	DSTC8		API Bank	
	Inference Speed (sec/sample)	Speed Up	Inference Speed (sec/sample)	Speed Up
Greedy Search	5.32	-	5.85	-
SCD Greedy Search	3.42	x1.56	3.33	x1.76
Beam Search	15.12	-	23.15	-
SCD Beam Search	6.33	x2.39	10.25	x2.25



Experiment Results – Inference Efficiency

Decoding Strategy	DSTC8		API Bank	
	Inference Speed (sec/sample)	Speed Up	Inference Speed (sec/sample)	Speed Up
Greedy Search	5.32	-	5.85	-
SCD Greedy Search	3.42	x1.56	3.33	x1.76
Beam Search	15.12	-	23.15	-
SCD Beam Search	6.33	x2.39	10.25	x2.25



Experiment Results – Inference Efficiency

Decoding Strategy	DSTC8		API Bank	
	Inference Speed (sec/sample)	Speed Up	Inference Speed (sec/sample)	Speed Up
Greedy Search	5.32	-	5.85	-
SCD Greedy Search	3.42	x1.56	3.33	x1.76
Beam Search	15.12	-	23.15	-
SCD Beam Search	6.33	x2.39	10.25	x2.25



Experiment Results – Context Efficiency

Setting	w. API Doc	w.o. API Doc	Δ
DSTC8 Unconstrained	37.63	33.74	-3.89
DSTC8 SCD	42.33	40.70	-1.63
API Bank Unconstrained	24.06	4.76	-19.3
API Bank SCD	56.64	22.81	-33.83



Experiment Results – Context Efficiency

Setting	w. API Doc	w.o. API Doc	Δ
DSTC8 Unconstrained	37.63	33.74	-3.89
DSTC8 SCD	42.33	40.70	-1.63
API Bank Unconstrained	24.06	4.76	-19.3
API Bank SCD	56.64	22.81	-33.83



Experiment Results – Context Efficiency

Setting	w. API Doc	w.o. API Doc	Δ
DSTC8 Unconstrained	37.63	33.74	-3.89
DSTC8 SCD	42.33	40.70	-1.63
API Bank Unconstrained	24.06	4.76	-19.3
API Bank SCD	56.64	22.81	-33.83



Conclusion

- ✓ We propose **State-tracked Constrained Decoding** that can effectively enforce constraints elicited from API Documentation, which yields **faithful generation**, **context efficiency**, and **better resilience** to API evolution
- ✓ We leverage **Constrained Token Search Trie** to reduce unnecessary forward inference passes, which yields **faster generation speed**
- ✓ We demonstrate the **effectiveness** of incorporating supervised signals with a small model to **discriminate and rerank** the beam-searched candidate generations of LLMs



Thank you!

**FANTASE - FANTastic SEquence and Where to Find Them:
Faithful and Efficient API Call Generation
through State-tracked Constrained Decoding and Reranking**

Stay Connected!

Email: wang@tamu.edu

LinkedIn: [Zhuoer Wang](#)

Homepage: edillower.github.io